Chapter 1

INTRODUCTION

The integration of Artificial Intelligence (AI) with Unmanned Aerial Vehicles (UAVs) has significantly enhanced their capabilities, enabling them to perform intelligent surveillance, autonomous monitoring, and real-time decision-making. However, despite advancements in drone technology, most low-cost UAVs still lack reliable, AI-powered object tracking systems. They depend heavily on manual control or basic vision-based techniques, which limit their performance in dynamic environments. To overcome these limitations, this project Auto Track: Smart Aerial Object Tracking with AI aims to develop a cost-effective aerial system capable of autonomously detecting, tracking, and following moving objects using deep learning and computer vision.

1.1 Problem Statement

Existing low-cost drones do not possess built-in intelligent tracking capabilities. They rely on manual navigation or simple colour-based detection, which fails in real-world conditions involving complex backgrounds, varying illumination, occlusions, and fast-moving targets. This creates operational challenges in areas such as surveillance, crowd monitoring, industrial inspection, and rescue missions where continuous and autonomous tracking is essential. Therefore, the problem addressed in this project is to design and implement an AI- driven aerial tracking system capable of performing real-time object detection and autonomous following using a lightweight drone platform.

1.2 Motivation

The motivation behind this project stems from the growing need for intelligent, automated monitoring systems that can reduce human workload and enhance operational efficiency. Manual drone operation is time-consuming, requires skill, and is unsuitable for long-duration or mission-critical applications. With the widespread adoption of AI and deep learning, it has become feasible to enable real-time object detection even on limited hardware. By integrating YOLO-based object detection with the DJI Tello drone, this project demonstrates how intelligent capabilities can be added to an economical and compact UAV, making autonomous tracking more accessible across various sectors.

1.3 Objectives

The key objectives of the project are:

1. To integrate the YOLO deep learning model with the drone's live video feed for real- time object detection.
2. To develop a robust tracking mechanism that accurately follows a selected target.
3. To design a control algorithm that converts detection output into autonomous drone movement.
4. To evaluate the system's tracking accuracy under different environmental and motion conditions.
5. To create a scalable and low-cost AI-based aerial tracking system suitable for practical applications.

1.4 Methodology

The methodology involves several structured stages:

1 **Dataset Preparation:** Capturing video footage using the Tello drone, extracting frames, and annotating them for model training.

2 **Model Training:** Training a lightweight YOLO model capable of running in real time on CPU-based systems.

3 **System Integration:** Streaming the drone's camera feed using OpenCV and performing YOLO inference on each frame.

4 **Tracking Algorithm:** Calculating the target's position relative to the frame Center and generating drone commands accordingly.

5 **Autonomous Control:** Using the dji-tellopy library to send movement commands (forward, backward, up, down, etc.) to maintain tracking.

6 **Testing and Evaluation:** Conducting indoor and outdoor tests to measure accuracy, latency, stability, and responsiveness.

1.5 Feasibility Study

- **Technical Feasibility:**

The DJI Tello drone supports programmable flight control and video streaming, making it suitable for implementing AI-based tracking. YOLO's lightweight architecture ensures real-time inference.

- **Operational Feasibility:**

The system requires minimal user intervention and is easy to operate, making it feasible for academic, research, and industrial use.

- **Economic Feasibility:**

The project is cost-effective as it relies primarily on open-source tools and the existing drone hardware, requiring no additional computing devices.

- **Schedule Feasibility:**

The project timeline is manageable, as tasks such as detection integration,

tracking implementation, and testing can be completed within an academic semester.

1.6 Team Distribution

The project was collaboratively executed by the four team members, with each member contributing to different aspects of design, development, testing, and documentation. The detailed work distribution is as follows:

1. **Chethana G M:**

- Initial drone setup
- Deciding the object tracking model
- Training the YOLO model
- Analyzing the object tracking dataset
- Implementing the trained model on the drone

2. **Dhanush M:**

- Procurement of the drone and essential hardware
- Handling publication activities related to the project
- analyzing the dataset
- Implementing the tracking model on the drone
- Preparing and maintaining project documentation

3. **Narein Karthik E:**

- Implementing the gesture control system for the drone
- Training the model for gesture-based operations
- analyzing the gesture control dataset
- Implementing gesture control functionalities on the drone

4. **Nithin P:**

- Designing and modelling the ducted propellers
- Testing the drone's performance with integrated systems
- Preparing documentation and report writing
- Training the gesture control model
- analyzing the dataset for gesture control.

Chapter 2

LITERATURE REVIEW

PAPER 1

[1] W. Giernacki *et al.*, "DJI Tello quadrotor as a platform for research and education in mobile robotics and control engineering," IEEE, 2022.

1. Introduction:

This paper provides an extensive evaluation of the DJI Tello, emphasizing its value as a low-cost, safe, lightweight, and versatile nano-UAV for research and educational purposes. The authors highlight that modern robotics and control engineering demand flexible, programmable UAV platforms that can support AI-based algorithms, computer vision tasks, and autonomous navigation experiments. Due to its stability, HD camera, optical flow sensor, and compatibility with Python SDK and ROS, the DJI Tello becomes an ideal testbed for students and researchers.

2. Limitations Identified in the Paper:

- Limited onboard computation
- Restricted flight time
- Indoor-only suitability
- Sensitivity to lighting changes for vision tasks
- SDK limitations for complex autonomous missions

3. Relevance to the Present Project (Auto Track):

This paper is highly relevant to Auto Track: Smart Aerial Object Tracking with AI because:

a. Validates Tello as a Research-Grade Platform

The study confirms that Tello can support computer vision, autonomous control, and AI processing—aligning perfectly with our object-tracking implementation.

b. Supports External AI Processing

The paper shows that Tello streams video externally for processing, validating our YOLO-based setup.

c. Demonstrates Real Projects Similar to Auto Track

Applications like face tracking and trajectory generation directly relate to our real-time object tracking goal.

d. Provides Structural Insights

The detailed UAV model and improved state machine help understand drone dynamics, which enhances our tracking Stability.
Thus, this paper provides both theoretical and practical foundations for designing and implementing Auto Track.

PAPER 2

[2] R. Fernandes *et al.*, "Enhancing image annotation with object tracking and image retrieval: A systematic review," IEEE, 2024.

1. Introduction:

This paper provides a detailed systematic review of two essential computer vision technologies object tracking and image retrieval and examines how they collectively enhance the quality, scalability, and automation of image annotation. With the exponential growth of visual data in fields such as surveillance, medical imaging, transportation, and autonomous systems, traditional manual annotation is no longer feasible due to high labour requirements, time consumption, and inconsistency. The authors emphasize that deep learning–powered tracking and retrieval now serve as core components of modern annotation pipelines, enabling accurate, semi-automated labeling across large datasets.

2. Strengths and Limitations Identified:

Strengths

- Improves annotation speed and reduces manual effort
- Enhances dataset consistency
- Supports both video and image annotation
- Leverages deep learning for better semantic understanding

Limitations

- Sensitivity to occlusion, blur, and changing appearance in tracking
- Semantic gap in retrieval
- Need for large, high-quality datasets
- Difficulty in fully automating annotation without human validation

3. Relevance to the Present Project (Auto Track):

This systematic review is highly relevant to Auto Track: Smart Aerial Object Tracking with AI because:

a. Strong Theoretical Foundation for Object Tracking

The paper's detailed coverage of modern tracking approaches reinforces the theoretical background of our drone-based YOLO tracking system. Addresses the Same Real-World Challenges like occlusions, lighting variation, fast movement, and background clutter directly correspond to issues faced by a moving UAV.

b. Supports Dataset Preparation for YOLO

Auto Track requires annotated datasets for training; the paper's insights on tracking- assisted annotation support efficient dataset creation.

c. Emphasizes Multi-Method Vision Systems

The combined approach of tracking + retrieval aligns with our future scope, where retrieval-based re-identification can enhance long-term drone tracking.

d. Confirms Feasibility of AI-Driven Annotation and Tracking

The review demonstrates that modern AI systems are capable of automating large-scale visual analysis, validating our project's design and objective of autonomous aerial tracking.

Thus, the insights from this paper meaningfully strengthen the conceptual foundation and technical justification of the Auto Track project.

PAPER 3

[3] Y. Qian *et al.*, "A multi-object tracking method in moving UAV based on IoU matching," IEEE, 2024

1. Introduction:

This paper proposes a specialized Multi-Object Tracking (MOT) method designed for moving UAV platforms, addressing the unique difficulties of aerial video analysis. UAV video streams suffer from unstable camera movements, altitude variation, fast- changing perspectives, and small object sizes. Traditional ground-based MOT systems fail under such conditions due to frequent occlusions, motion blur, and inconsistent detection.

The authors introduce an improved IOU (Intersection over Union)–based matching method optimized for real-time tracking on UAVs, enabling accurate and lightweight multi-target tracking suitable for airborne applications.

2. Strengths and Limitations:

Strengths

- Lightweight and computationally efficient
- Effective under UAV motion disturbances
- Low identity-switch rate due to trajectory buffering
- Compatible with any fast detector

Limitations

- IOU matching struggles during heavy occlusions
- Highly dependent on detector accuracy
- No re-identification (Re-ID) mechanism for look-alike objects

3. Relevance to the Present Project (Auto Track):

This paper is highly relevant to Auto Track: Smart Aerial Object Tracking with AI, providing several key insights:

a. Addresses UAV-Specific Tracking Challenges

The issues identified—small object sizes, dynamic background, rapid motion—are the same constraints faced by the DJI Tello drone in our project.

b. Lightweight Tracking Approach Fits Tello's Limitations

The IOU-based method supports real-time operation without heavy GPU usage, aligning well with our external YOLO inference pipeline.

c. Supports Frame-by-Frame YOLO Detection

The discussed tracking-by-detection workflow reinforces our design choice to detect objects each frame and compute positional error for drone motion.

d. Future Expansion into Multi-Object Tracking

Auto Track currently supports single-object tracking; this paper provides a theoretical basis for extending it into multi-target tracking.

e. Emphasizes Importance of Robust Data Association

Even though Auto Track tracks a single target, the overflow handling and identity- preservation strategies help during momentary detection loss caused by drone motion.

Thus, this study directly strengthens the methodological foundation for building a robust, real-time UAV object tracking system like Auto Track.

PAPER 4

[4] M. M. Zaidi *et al.*, "Suspicious human activity recognition from surveillance videos using deep learning," IEEE, 2024.

1. Introduction:

This paper presents a comprehensive deep-learning framework for Suspicious Human Activity Recognition (SHAR) in surveillance environments. With the rapid expansion of video surveillance systems, manual monitoring has become impractical due to the volume of data and the need for timely detection of threats. The authors aim to automate the recognition of six critical suspicious activities Running, Punching, Falling, Snatching, Kicking, and Shooting using advanced deep-learning models. The work highlights the increasing need for automated surveillance solutions due to rising security concerns in public places, transportation hubs, and high-risk areas.

2. Strengths and Limitations:

Strengths

- Custom dataset tailored for surveillance contexts
- Strong multi-model comparison
- High performance on real-world videos
- Balanced treatment of spatial and temporal learning

Limitations

- Dataset diversity remains limited to six activities
- Real-time deployment considerations are not deeply explored
- Temporal consistency varies across different model types

3. Relevance to the Present Project (Auto Track):

This paper significantly aligns with and strengthens the methodology for Auto Track: Smart Aerial Object Tracking with AI, in several ways:

a. Importance of Spatio-Temporal Learning

The drone-based tracking system in Auto Track deals with continuous video streams similar to SHAR. Lessons on frame extraction, temporal sequence modeling, and pre- processing directly support reliable tracking.

b. Dataset Handling and Annotation Strategy

The systematic approach of creating and annotating a custom dataset provides a strong reference for curating datasets for drone-based object-following and gesture recognition modules.

c. Multi-Model Comparison Framework

Evaluating multiple architectures is relevant for Auto Track, especially in selecting lightweight yet accurate models suitable for real-time drone inference (e.g., YOLO + tracking-by-detection).

d. Video-Based Prediction and Real-Time Insights

Just as the SHAR system predicts actions in real-world videos, AutoTrack uses continuous detection and tracking to maintain lock on targets during drone movement.

e. Enhancing Reliability in Complex Scenarios

The SHAR paper's focus on occlusions, viewpoint changes, and temporal inconsistencies parallels the challenges faced by UAV video feeds, improving understanding of solutions for robust aerial tracking.

Thus, this study provides methodological knowledge that supports both the object tracking and gesture-control components of the Auto Track system.

PAPER 5

[5] T. Keawboontan and M. Thammawichai, "Toward real-time UAV multi-target tracking using joint detection and tracking," IEEE, 2023.

1. Introduction:

This paper presents a unified, real-time framework for multi-target tracking (MTT) in Unmanned Aerial Vehicle (UAV) video streams. With drones being increasingly deployed in surveillance, environmental monitoring, crowd analysis, and defense operations, robust tracking of multiple objects becomes essential.

However, UAV footage poses unique challenges: rapid camera movement, altitude variation, small object sizes, occlusions, and cluttered backgrounds. The authors propose a Joint Detection and Tracking (JDT) approach to tackle these issues, combining object detection and temporal data association into a single, efficient pipeline.

The study aims to increase tracking accuracy and stability while ensuring real-time performance suitable for resource-constrained aerial platforms.

2. Strengths and Limitations:

Strengths

- Real-time performance suitable for UAV operation
- Accurate identity maintenance across complex trajectories
- Unified model reduces computational overhead
- High robustness to dynamic aerial environments

Limitations

- Dependent on the quality of object detection outputs
- Multi-target tracking increases complexity and may not generalize well in extremely dense environment.

3. Relevance to the Present Project (Auto Track):

This paper directly informs and strengthens the development of Auto Track: Smart Aerial Object Tracking with AI, in several important ways:

a. Reinforces the Tracking-by-Detection Approach

Auto Track uses YOLO for real-time detection, similar to the JDT architecture. The paper validates that integrating detection and tracking is the most effective strategy for aerial applications.

b. Addresses UAV-Specific Tracking Challenges

Issues such as camera shake, small objects, and occlusions are identical to the conditions encountered by the DJI Tello drone. The solutions discussed (IOU matching, Kalman filtering, embedding-based association) can guide enhancements in Auto Track.

c. Provides Insights for Scaling to Multi-Object Tracking

Auto Track currently tracks a single object. This paper provides the theoretical foundation to extend it to multiple targets, an important future scope.

d. Emphasizes Efficiency for Real-Time Flight

Since the Tello drone has limited onboard compute, the lightweight, optimized nature of JDT is particularly relevant for sustaining smooth tracking and stable flight adjustments.

e. Enhances Stability in Gesture-Control Module

The motion prediction and temporal consistency methods discussed can also improve the reliability of gesture recognition when the drone follows dynamic hand movements.

Overall, this paper offers strong methodological guidance for building a reliable, real-time aerial tracking system, making it highly relevant to the Auto Track project.

PAPER 6

[6] Z. Zhang *et al.*, "AutoFilmer: Autonomous aerial videography under human interaction," IEEE, 2023.

(Department of Electronic and Information Engineering, The Hong Kong Polytechnic University)

1.Introduction

This paper presents the design and development of an autonomous quadcopter-based camera operator, intended to function as an automatic cameraman for live outdoor news reporting. The system uses a quadcopter equipped with a camera, onboard processing unit, sensors, and customized tracking algorithms to continuously maintain a stable view of a reporter holding a microphone fitted with distinct color markers. The drone adapts its movements

forward/backward, rotation, and lateral shifts—to keep the reporter centered in its field of view at a fixed distance.

2 Limitations Identified

- Tracking relies entirely on colour markers (not robust to lighting changes).
- The system cannot operate with multiple human subjects.
- No deep-learning–based detection (purely classical image processing).
- Distance estimation is approximate (no depth sensor).
- Flight stability is affected by environmental factors (wind, shadows).
- No gesture control or autonomous cinematic path planning.

3. Relevance to the Present Project

This paper directly supports and strengthens your project Auto Track: Smart Aerial Object Tracking with AI, in the following ways:

a. Tracking Concepts

Their marker-based tracking validates the importance of real-time visual feedback loops, which your project enhances using YOLOv8 deep learning for robust face/body detection.

b. Control Mechanism

Their PID-based movement control is closely aligned with your approach of computing:

- Pitch
- Yaw
- Roll
- Altitude

based on target position and bounding box error.

c. Obstacle Avoidance

The method of overriding tracking with safety-based decisions can be integrated in yoursystem (e.g., Tello’s IR sensor data).

d. Tello-Based Implementation Feasibility

Their hardware-software architecture validates:

- Python backend
- Real-time image capture
- Command-based control All of which are also used in your drone.

e. Justification for Ducted Propellers

- Their system suffered from pop wash affecting IR sensors. Your project addresses stability
- enhancements via ducted propellers, which is a clear technological improvement.

Paper 7

[7] G. D. Ramaraj, S. Venkatakrishnan, G. A. Balasubramanian, and S. Sridhar, "Aerial surveillance of public areas with autonomous track and follow using image processing," (Department of Electrical and Electronics Engineering, Sri Sairam Engineering College, Chennai, India)

1. Introduction

The paper discusses the development of an autonomous aerial surveillance system using quadcopters, aimed at improving public safety and overcoming the shortcomings of traditional Closed-Circuit Television (CCTV) systems. The authors argue that fixed CCTV cameras fail to provide real-time, dynamic tracking of human activities, especially in large open areas.

To address this, the paper proposes an image-processing-driven autonomous drone capable of detecting, tracking, and following human targets in public spaces using advanced filtering and tracking algorithms.

2. Limitations Identified

Despite strong capabilities, limitations include:

- Complex computation cost for PHD–MCMC
- Environmental challenges (lighting, occlusion)
- Limited real-world deployment scenarios
- No deep-learning–based human detection (relies on classical filtering)
- Heavy reliance on GPS and tracking markers for autonomous landing
- No dynamic obstacle avoidance capability

3. Relevance to the Present Project

This paper directly supports our **Auto Track: Smart Aerial Object Tracking with AI** in several ways:

a . Reinforcement of Autonomous Tracking Concepts

Their use of PHD–MCMC confirms the need for: Reliable multi-target tracking. Our project improves upon this by using YOLOv8, which handles detection and tracking with higher
accuracy and lower computational complexity.

b. Swarm and Coverage Ideas

Even though your project uses a single drone (DJI Tello), the surveillance concepts provide justification for future extensions (Chapter 11 – Future Enhancement):

- Multi-drone tracking
- Extended coverage
- AI-assisted coordination

c. Importance of Energy & Stability Features

The paper highlights energy-efficient planning and charging automation, validating:

- Your integration of ducted propellers for stability
- Potential integration of energy-saving algorithms

d. Relevance to Real-Time, Real-World Application

The paper's focus on real-world surveillance strengthens the real-time utility of your system:

- Your drone tracks humans using face/body detection
- Your control system computes pitch–yaw–roll commands
- Your YOLO models outperform classical algorithms used in the paper

Paper 8

[8] R. Gadre, A. Goyal, S. Khanna, S. Jain, and V. Saligrama, "Auto Filmer: Autonomous aerial videography under human interaction," IEEE, 2023.

1. Introduction

The paper "Auto Filmer: Autonomous Aerial Videography Under Human Interaction" presents a drone-based autonomous cinematography system designed to record humans intelligently without requiring manual piloting. This system addresses the growing need for hands-free aerial videography in robotics, entertainment, sports, and surveillance

The key motivation is to allow human subjects to interact naturally with the drone, enabling intelligent behavior such as following, framing, understanding commands, and maintaining cinematic shot stability.

2. Limitations

The system exhibited limitations such as:

- Sensitivity to poor lighting
- Reduced accuracy during fast movements
- Gesture recognition failures under occlusion
- No deep learning models for robust tracking
- Limited indoor performance due to ultrasonic sensing constraints

3. Relevance to the Present Project (Auto Track)

This paper is highly relevant to your drone project in the following ways:

a. Cinematic Filming Concepts

Auto Track's goal of stable tracking aligns with Auto-Filmer's:

- Smooth trajectory planning
- Stable framing
- Human-oriented camera control

This supports your control and stability enhancements using ducted propellers.

b. Human Pose Detection vs YOLOv8

Their classical pose detection pipeline shows the limitations of older approaches. Your project surpasses this by using **YOLOv8**, which provides:

- More robust detection
- Faster inference
- Higher tracking accuracy

c. Control Architecture Inspiration

Both systems use:

- Feedback loops
- Real-time adjustments
- Hierarchical control

d. Reinforces Stability Needs

The paper's issues with jerkiness justify:

- Your use of ducted propellers
- Your refined yaw–pitch–roll PID mapping
- Your closed-loop stable control approach

Paper 9

[9] H. Fakhrurroja *et al.*, "Automated license plate detection and recognition using YOLOv8 and OCR with Tello drone camera," IEEE, 2023.

1. Introduction

This paper presents an autonomous license plate detection and recognition system using the DJI Tello drone coupled with YOLOv8 object detection and Optical Character Recognition (OCR). The study explores the use of low-cost drones for traffic monitoring, surveillance, and vehicle identification.

The system aims to automatically detect vehicles, extract license plate regions, and recognize plate characters in real time using deep-learning techniques.

2. Limitations

OCR errors occur with:

- Motion blur
- Strong glare
- Steep camera angles
- Limited night-time detection
- Tello's low-resolution camera restricts accuracy
- No advanced tracking—only detection + recognition
- No mechanical stability enhancement (your project's ducted propellers provide this)

1. Relevance to the Present Project (Auto Track)

This paper is highly relevant to project for the following reasons:

a. Demonstrates the Capability of YOLOv8 With Tello

Confirms that Tello + YOLOv8 is feasible for real-time detection, validating your use of YOLOv8 for person tracking.

b. Similar Real-Time Processing Pipeline

Both systems use:

- Tello video stream
- Python + OpenCV
- Fast object detection
- Autonomous decision-making

This directly aligns with Auto Track's backend architecture.

c. Modular Architecture

Their pipeline structure (Detection → Crop → OCR) resembles your tracking pipeline (Detection → Error Computation → Control), confirming that modular design is ideal for UAV systems.

PAPER 10

[10] J. Kim *et al.*, "Deep learning based malicious drone detection using acoustic and image data," IEEE, 2022.

1. Introduction

With the rapid growth of drone usage, malicious UAV activities have become a substantial security threat across airports, military bases, public gatherings, and private zones.

Traditional radar-based drone detection systems are expensive, heavy, and limited by line- of-sight constraints.

The paper proposes a deep-learning–based detection model that combines acoustic signals and visual image data to detect malicious drones efficiently. This multi-modal approach aims to increase detection accuracy and reliability in real-world scenarios.

2. Limitations

- Reduced performance in extremely noisy environments
- Works best when both audio and visual data are available
- Requires high-quality microphones and multiple cameras for full coverage
- Night-time visual detection still weak
- Not optimized for long-distance detection

3. Relevance to the Present Project (Auto Track)

This paper contributes to our Auto Track drone project in several valuable ways:

a. Deep Learning on UAV Camera Feeds

The paper demonstrates the effectiveness of **CNN/YOLO models** on drone-collected image data—validating your choice of **YOLOv8** for tracking.

b. Use of Multi-Modal Vision Concepts

While your project uses RGB video, this paper extends vision with acoustic inputs. This can be a future enhancement for:

- Collision detection

- Human activity recognition
- Multi-sensor stability control

c. Importance of Dataset Quality

The study emphasizes:

- Proper data collection
- Balanced training dataset
- Variations in environment

This strengthens your dataset preparation section (Chapter 7 – Dataset).

d. Relevance to Drone Security & Safety

Your drone can incorporate:

- Threat detection logic
- Safety protocols
- Autonomous behavioral responses

This aligns with project's real-world use cases.

e. Reinforces Deep Learning Advantage Over Classical CV

The paper's results show deep learning outperforms traditional CV—justifying:

- YOLOv8 for tracking

PAPER 11

[11] M. Otte, A. Pandey, T. L. Molloy, R. Holzapfel, and J. W. Roberts, "DJI Tello quadrotor as a platform for research and education in mobile robotics and control engineering," IEEE

1. Introduction

This paper evaluates the DJI Tello quadrotor drone as an affordable and accessible research platform for robotics, computer vision, and control engineering. With many universities shifting to low-cost yet high-capability drones for laboratory teaching and student projects, the authors analyze Tello's suitability for:

- Control algorithm development
- Computer vision testing
- Autonomous navigation experiments
- Educational robotics

The paper positions Tello as a **bridge between simulation and advanced UAV platforms**, making it ideal for rapid prototyping.

2. Limitations

The limitations highlighted are:

- No obstacle detection sensors
- Wi-Fi signal instability over long distances
- Limited payload capacity
- Video stream compression reduces image clarity
- SDK restricts access to raw sensor data
- Not suitable for outdoor/windy environments
- No onboard AI processing

Despite these constraints, Tello is still highly effective for **indoor AI and control research**.

3. Relevance to the Present Project (Auto Track)

This paper is **directly relevant** to your project for several reasons:

a. Confirms Tello as a Reliable Research Platform

The paper proves Tello is suitable for:

- AI-based tracking
- Python control
- Vision-based flight

b. Supports YOLOv8 Real-Time Video Processing

Although the paper used classical computer vision, its results show Tello's **video stream is stable and fast enough** for deep-learning inference matching your training pipeline.

a. Reinforces Need for Ducted Propellers

The paper mentions Tello's lightweight frame makes it sensitive to disturbances, which supports your use of ducted propellers to stabilize flight.

b. Aligns with Your Control System Design

Their PID-based control experiments align with:

- Your closed-loop tracking
- Real-time error correction
- Stable hovering + tracking

c. Validates Offboard Processing Architecture

The paper confirms best practices:

- Run YOLOv8 on laptop
- Send control commands to Tello

- Use Python + SDK
- Maintain stable 25 FPS

This matches your entire backend design.

PAPER 12

[12] Jhonatan P. Cambisaca, Henry M. Castro, Henry D. Maldonado, Pablo Barbecho Bautista, "LoRa-Enabled Communication System for Autonomous DJI Tello Drone Operation", *2025 IEEE*

1. Introduction

The paper *"*Exploring DJI Tello as an Affordable Drone Solution for Research and Education*"* investigates the potential of the **DJI Tello EDU** as a low-cost platform for robotics, AI, trajectory execution, and computer vision tasks. The authors aim to assess whether an inexpensive drone can reliably support research-oriented and educational activities, especially in environments where safety, cost limitations, and accessibility are major considerations.

The study demonstrates Tello's capabilities through two major case studies—object detection & tracking, and autonomous trajectory execution using an external localization system (Opti Track).

3. Limitations

- Tello's built-in camera is low-resolution and not suitable for long-distance visual tasks.
- No onboard high-power processor for embedded AI.
- Reliant on off-board computing, causing network delay.
- No accurate internal localization → requires external systems.
- Limited outdoor usability due to drift and Wi-Fi control.

4. Relevance to the Present Project (Auto Track)

This paper directly strengthens and validates our work in several ways:

Confirms Tello as an Effective AI/Tracking Research PlatformShows Tello is reliable for YOLO-based tracking—mirroring your Auto Track architecture.

a. Supports Your Use of YOLOv8

The authors use YOLO for real-time detection, proving that deep learning frameworks integrate well with Tello.

b. Reinforces Need for Enhanced Stability → Your Ducted Propeller Upgrade

The paper notes instability due to hardware limitations. Your ducted propeller system directly addresses:

- Drift
- Hover instability

- Wind sensitivity
- Tracking errors

c. Validates Offboard Python + Tello SDK Architecture

The system described in the paper is almost identical to your implementation:

- Python backend
- Real-time computer vision
- UDP command control
- Offboard inference pipeline

This confirms industrial and research alignment.

d. Encourages Future Upgrades

The paper's trajectory accuracy improvement using Opti Track supports:

- Future integration of motion capture

PAPER 13

[13] I. E. Al-Shayeb *et al.*, "Integrating AI-driven robust control algorithm with 3D hand gesture recognition to track an underactuated quadrotor UAV," IEEE, 2024.

1. Introduction

This paper proposes a robust AI-driven control framework combined with 3D hand gesture recognition to control and track an underactuated UAV. The system integrates:

- Advanced control algorithms (nonlinear robust control)
- Real-time gesture commands
- Two-arm human simulation for intuitive drone interaction

The study focuses on improving human–drone interaction (HDI) and achieving stable tracking even under uncertainties, disturbances, and nonlinear UAV dynamics.

2. Limitations

The authors identify several limitations:

- Tested only in **simulation**, not on physical drones
- Requires depth cameras, which may not be available on small UAVs
- Gesture recognition accuracy depends on lighting and camera angle
- Robust controller tuning is computationally intensive
- Limited to predefined gesture sets

Despite these limitations, the system shows excellent potential for real-world applications.

3. Relevance to the Present Project (Auto Track)

This paper is directly relevant to project for several reasons:

a. Strong Support for Gesture-Based Drone Control

Your project uses gesture recognition for drone commands, which aligns perfectly with this paper's approach.

It validates your decision to integrate:

- 3D gesture commands
- AI-based interaction
- Real-time user-driven control

b. Stability Enhancement Matches Your Ducted Propeller Upgrade

Their robust controller is designed to manage disturbances. You achieve a similar improvement physically through ducted propellers, a simpler yet effective stability method.

c. Real-Time Human–Drone Interaction Framework

The human–drone communication model in the paper supports:

- Autonomous tracking
- Real-time adjustments
- Smooth control execution

All of which are present in your Auto Track system.

d. Reinforces Need for AI-Based Control

Our system uses YOLOv8 for tracking. This paper supports the need for AI-assisted control and interaction.

e. Architecture Similarity

Their three-module system (gesture → AI controller → UAV) parallels your architecture:
Detection Module → Tracking Module → Control Module

PAPER 14

[14] G. Sathish Kumar and A. K. Damodaran, "Machine learning-based classification of ducted and non-ducted propeller type quadcopter," IEEE.

1. Introduction

This paper investigates the classification of ducted vs non-ducted quadcopters using machine learning algorithms. The presence of ducted propellers significantly influences:

- Thrust efficiency

- Noise characteristics
- Vibration patterns
- Flight stability
- Power consumption

The study collects sensor-based flight data to train machine learning models that differentiate between the two propeller configurations. This paper is extremely relevant to your project because you integrate ducted propellers for improved flight stability.

2. Limitations

- Limited dataset size
- Only one model of quadcopter tested
- ML models require large sensor data for optimal accuracy

Despite these limitations, results strongly support the aerodynamic benefits of ducted propellers.

3. Relevance to the Present Project (Auto Track)

This paper is **highly relevant** because your project includes **ducted propellers for stability enhancement**.

a. Confirms Ducted Propellers Improve Stability

Findings validate your integration of ducted propellers, especially for:

- Reduced vibration → better tracking
- More stable video feed → improved YOLOv8 accuracy
- Smoother PID control

b. Supports Aerodynamic Benefits

Ducted propellers:

- Reduce tip vortices
- Increase thrust efficiency
- Improve safety by shielding blades

This directly supports your mechanical design decisions.

c. Enhances Flight Control Loop

Lower vibration leads to:

- More accurate IMU readings
- Better positional stability
- Smoother object following

Which are all critical for Auto Track's closed-loop tracking system.

d. Provides Scientific Validation for Your Design**

You can cite this paper in your report to justify:

- Why ducted propellers are used
- Performance improvement after integration

PAPER 15

[15]T. Keawboontan and M. Thammawichai, "Toward real-time UAV multi-target tracking using joint detection and tracking," IEEE, 2023.

1. Introduction

This research proposes a real-time multi-object tracking (MOT) system designed specifically for UAV platforms. The approach integrates joint detection and tracking into a single architecture, overcoming the limitations of classical multi-target tracking pipelines that run detectors and trackers separately.

Because UAV videos often suffer from motion blur, sudden movements, occlusions, and scale variations, typical MOT algorithms fail to maintain object IDs and real-time performance. The authors address these challenges using a deep-learning-based architecture capable of processing adjacent frames and predicting positions more efficiently.

2. Limitations

The paper acknowledges a few limitations:

- Difficulty tracking very tiny objects in cluttered scenes
- Some failures in low-light environments
- Performance depends on object movement speed
- Needs improvement for dense scenes with overlapping targets

Despite these limitations, the approach significantly advances real-time UAV MOT.

3. Relevance to the Present Project (Auto Track)

This paper is highly relevant to your project for several reasons:

a. Validates Joint Detection + Tracking (same as YOLOv8 tracking)

Your Auto Track system uses YOLOv8 for detection + tracking, following the same joint architecture.

b. Supports UAV-specific optimizations

The paper deals with:

- Motion blur

- Vibration
- Sudden movement

Your project solves these mechanically by adding ducted propellers, improving stability

aligned with the paper's emphasis on stable motion for better tracking

Your drone tracks a single primary target, but the multi-target improvements directly benefit single target stability as well.

c. Helps justify design choices in Chapter 3 and 5

The paper strengthens arguments related to:

- Why joint detection-tracking is better
- Why lightweight neural networks help in Tello-based systems
- How temporal pairing improves tracking

PAPER 16

[16] Z. Chen *et al.*, "MaskTrack: Auto-labeling and stable tracking for video object segmentation," IEEE, 2025.

1. Introduction

This paper introduces Mask Track, a new framework for video object segmentation (VOS) that focuses on two key challenges:

- the high cost of dense video mask annotation, and
- unstable instance tracking in long, complex videos.

The authors propose a zero-shot auto-labeling strategy built on the Segment Anything Model (SAM) and a transformer-based VOS architecture that combines pixel-level memory (PM) and instance-level memory (IM) to achieve stable long-term tracking and segmentation.

2. Limitations

- Auto-labeling is computationally expensive because SAM must run on every frame.
- Distinguishing highly similar, closely moving instances remains challenging.
- The framework currently uses only RGB information; depth or other modalities are not yet exploited.

3. Relevance to Your Project (Auto Track)

- Shows how long-term stable tracking can be improved using instance-level memory, which conceptually supports your aim of robust person tracking over longer drone flights.
- The auto-labeling strategy is relevant for creating synthetic or augmented datasets for your YOLOv8 training, especially if you capture your own drone videos and want automatic annotations.
- Mask Track's success in dense, crowded scenes provides ideas for extending Auto Track from single-person following to multi-person tracking in the future.

PAPER 17

[17]Q. Li *et al.*, "Multi-target tracking method for drone swarm based on interaction feature extraction," IEEE, 2024.

1. Introduction

This paper addresses multi-target tracking (MTT) for infrared drone swarms, where multiple small drones move in groups with frequent occlusions and platform shaking. The authors propose a method that focuses on interaction features between targets, using Graph Convolutional Networks (GCN) and Graph Attention Networks (GAT) to aggregate and refine trajectory information.

2. Limitations

- Designed specifically for infrared swarm data; may not directly generalize to RGB or single-drone scenarios.
- Relies on accurate initial detections; detection failures propagate into tracking errors.
- Interaction features may be less informative when swarm density is low.

3. Relevance to Your Project (Auto Track)

- Introduces the idea of using interaction/motion features and graph-based models for tracking, which could be useful if you extend AutoTrack to multiple people or multiple drones.
- The use of Gaussian smoothing and graph-based refinement provides inspiration for further stabilizing trajectories in your YOLOv8-based tracker.

PAPER 18

[18] S. Jang *et al.*, "Prediction-based auto-pilot interface for drone-to-object chasing using historical TSPI data," IEEE, 2023.

1. Introduction

This paper focuses on anti-drone applications, proposing a system where a "good" drone chases a target drone by predicting its future trajectory using Time-Space Position Information (TSPI) and machine-learning-based motion prediction. The predicted positions are fed into the DJI Auto-Pilot API to generate trajectory commands automatically.

2. Limitations

- Experiments are largely simulation-based; real-world disturbances (wind, GPS noise) may reduce accuracy.
- Requires reliable external TSPI sensors (e.g., radar) which may not be available in small-scale setups.
- Prediction quality decreases for highly maneuverable or erratic targets.

3. Relevance to Your Project (Auto Track)

- Shows how prediction of future target positions can improve tracking robustness—this can inspire future enhancements where Auto Track predicts the next location of a walking/running person for smoother control.
- Uses DJI flight APIs and converts pitch/roll/throttle into position changes, similar to your control mapping from YOLOv8 error to Tello commands.

Reinforces the importance of considering latency in the control loop, which is also critical in your real-time tracking pipeline.

PAPER 19

[19] K. Xu, G. Zheng, and F. Zhang, "Deep learning + Kalman filter for industrial coil center tracking," IEEE, 2022.

1. Introduction

This paper presents a computer-vision algorithm that uses deep learning and Kalman filtering to detect and track steel sheet coils being transported to an uncoiler in a cold rolling mill. Accurate tracking of the coil-hole center is crucial for an automatic system that positions the coil correctly on the uncoiler without human intervention.

2. Limitations

- The method is specialized for a single object type (coil hole) and a constrained motion pattern.
- Requires a labeled dataset for CNN training.
- Performance is tied to fixed camera geometry; major camera shifts would require retraining or recalibration.

3. Relevance to Your Project (Auto Track)

- Demonstrates a deep-learning + Kalman filter pipeline for robust tracking, similar to how you could fuse YOLOv8 detections with a Kalman filter to smooth drone tracking of people.
- Shows that monocular vision can replace more complex sensors in harsh industrial environments—analogous to using Tello's single RGB camera for object tracking.
- The long-term deployment (4+ months) illustrates the importance of robustness to lighting changes and noise, which is also critical for your outdoor/indoor drone tracking.

2.2 COMPARITIVE TABLE

Table 2.1 COMPARATIVE TABLE

Paper No./ Title/ First Author	Problem	Objective	Methodology	Results	Relevance to Our Auto Track Project
P1 – DJI Tello Quadrotor as a Platform for Robotics & Control	Costly education aldrones, limited safe indoor UAVs	EvaluateTello for robotics research	Hardware eval, SDK tests, CV experiments	Stablehover, suitable for CV & control	Validates using Tello for YOLOv8 tracking project
P2 – Enhancing Image Annotation Using Tracking& Retrieval	Manual annotatio n costly	Automate annotationusing tracking	Tracking+ similarity-based image retrieval	Better annotation speed & consistency	Supports dataset creation for YOLOv8 training
P3 – Auto Filmer: Autonomous Aerial Videography	Hard to manually film humans	Buildautonomous filming drone	Pose estimation +trajectory planning	Smooth autonomous filming	Supports gesture control& smooth tracking design
P4 – License Plate Detection Using YOLOv8	Hard to detect platesvia drone	Real-time ALPR from Tello	YOLOv8 + Tesseract OCR	100% detection, 66% OCR accuracy	Supports YOLOv8 real-time drone detection feasibility

P5 – Malicious Drone Detection Using Acoustic & Image Data	Hardto detect small drones	Combinea udio + vision deep learning	CNN + YOLO + sensor fusion	High accuracy withfusion	Supports multi-modal ideas for future upgrades
P6 – Exploring DJI Tello for Research& Education	Lack of low-cost teaching UAVs	AnalyzeT ellofor lab usage	Control tests,CV tasks	Goodindoor stability	Confirms Tello is ideal for AutoTrac k hardware
P7 – Systematic Survey of Drone Software Engineering	Lack of unified SE overview	Providesu rveyof drone software stack	Reviewof control, vision, autonomy	Comprehens ive taxonomy	Helps justify complete system architectu re
P8 – Image Annotation & Object Tracking Systematic Review	Difficulty in consistent tracking	Review of segmentati on/trac king methods	Comparati v e analysis	Identifies best-performing trackers	Supports YOLO choice for improved tracking
P10– DCE-YOLOv8 forDrone Vision	YOLOv8 too heavy	Createligh tweight version	Depthwise conv + channel enhanceme nt	Higher speed, comparable accuracy	Supports real-time YOLO optimizati on insight
P11 – Suspicious Human Activity Recognition	Hard to detect suspicious behavior	Deeplearn ing for surveillan ce	CNN+ LSTM	Good accuracy	Human behavior analysis future extension

P12 – Joint Detection& Tracking for UAV	Fragmented detect+track pipeline	Merge detection+ tracking	FCOS + memory networks	77 FPS, high MOTA	Confirms YOLOv8 tracking approach
P13 – MaskTrack : Auto Labeling& Stable VOS	Annotation cost & ID drift	Auto-labeling+ stable tracking	SAM + transformer -based memory	SOTA segmentation performance	Usefulfor dataset generation
P14 – Drone Swarm Tracking via Interaction Features	Infrared drone-swarm occlusion	Motion-only features forMTT	GCN+ GAT + Gaussian smoothing	84.9% MOTA	Usefulfor multi- target future work
P15 – Prediction- Based Drone Chasing (TSPI)	Hard to chase moving drone	Predict future UAVpositions	ML prediction + DJI autopilot	Successful chasing in simulation	Inspires predictive motion control
P16 – Steel Coil Tracking Using DL + Kalman Filter	Classical CV fails in factory	DL + KF for robusttracking	CNN + template matching+ KF	Robust 4-month deployment	KF-based smoothing applicable to AutoTrac k
P17 – Auto Camerama n Quadcopter System	Hard to self-film with drones	Autonomous cameraoperator	CV-based pose recognition	Smooth filming	Supports person-following idea
P18 – Aerial Surveillance Track- and-Follow	Basic tracking fails in crowds	Followtarget using CV	Color + contour+ centroid	Worksin simple scenes	Shows why deep learningis needed

P19 – YOLO-based Object Detection Survey	Many YOLO variants unclear	Compare YOLO methods	Benchmark analysis	Listsbest YOLO versions	Supports selection of YOLOv8

Chapter 3

SYSTEM ANALYSIS

3.1 Overview

The Auto Track: Smart Aerial Object Tracking with AI system is designed to enable fully autonomous tracking of a chosen object using the DJI Ryze Tello drone. Unlike conventional drones that rely heavily on manual navigation, Auto Track integrates YOLO- based real- time object detection, vision-driven navigation, and a newly added ducted propeller module that improves aerodynamic stability during flight.

The system architecture is hybrid in nature:

- The drone provides flight stabilization and video streaming.
- The external laptop performs AI inference and control logic.
- The ducted propellers mechanically enhance airflow stability and reduce lateral drift during tracking.

This combined approach makes Auto Track capable of precise, safe, and stable autonomous tracking indoors and in controlled outdoor environments.

3.2 Existing System

Existing low-cost UAV systems suffer from multiple limitations that make autonomous object tracking extremely difficult:

1. Manual Control:

The operator must continuously steer the drone to keep the target within the frame. This results in:

- High operator workload
- Inefficient tracking
- Poor accuracy in fast-moving scenes

2. Basic Vision Techniques:

Some drones use simple threshold-based or optical-flow-based tracking, which suffer from:

- Poor performance under lighting variations
- Failure when the object changes size or orientation
- Inability to recover after occlusion

3. No Intelligent Recognition:

Existing systems cannot:

- Identify targets (person, vehicle, object)
- Track based on object class
- Perform collision-aware navigation

4. No Autonomy:

Manual intervention is required throughout the flight, making long-duration surveillance impractical.

Drawbacks of Existing Systems

- No real-time object detection
- No dynamic tracking
- Weak environmental adaptability
- No stable alignment control
- Fails under occlusion, motion blur, and complex backgrounds

Overall, existing low-cost UAV systems are **operator-dependent**, **inaccurate**, and **unsuitable for intelligent monitoring tasks**.

3.3 Proposed System

The proposed **Auto Track system** uses AI-powered object detection and vision-based control to achieve real-time autonomous tracking.

1. YOLO-Based Object Detection:

- High-accuracy detection at 25–30 FPS
- Identifies object class and bounding box
- Robust to lighting variations and partial occlusions

2. Drone Video Streaming (OpenCV):

- Real-time frame acquisition
- Frame-by-frame analysis

3. Position & Error Calculation:

- Computes difference between bounding box center and frame center
- Determines direction: left, right, up, down, forward, backward

4. Autonomous Drone Control (djitellopy API):

- Converts positional error into drone movements
- Performs yaw rotation, lateral correction, and distance control
- Ensures stable tracking loop

5. Hybrid Architecture:

- Drone handles flight stabilization
- Laptop handles YOLO inference + control logic

6. Gesture Control Extension (Optional Module):

- Recognizes user hand gestures for controlling the drone
- Integrated using CNN + temporal models

7. Ducted Propeller Integration:

Custom 3D-printed ducted propellers significantly enhance:

- Hover stability
- Yaw control precision
- Airflow direction and thrust efficiency
- Safety (reduced risk of propeller contact)
- Resistance to minor turbulence

This makes the tracking smoother and improves YOLO detection reliability by reducing camera shake.

3.4 Comparison Between Existing System & Proposed System (in Table 3.1)

Table 3.1--comparative table

Feature	Existing System	Proposed Auto Track System
Tracking Type	Manual / basic vision	AI-based autonomous tracking
Detection Method	None or simple threshold	YOLO deep learning model
Stability	Moderate to poor	Improved with ducted propellers
Occlusion Handling	Failswhentargetis blocked	YOLO re-detection + error correction

Target Identification	Not possible	Detects object class & tracks specific target
User Effort	Continuousmanual control	Minimal user involvement
Accuracy	Low	High (AI inference at 25–30 FPS)
Hardware Cost	Moderate	Still low—uses existing laptop GPU/CPU
Robustness	Sensitive to lighting& motion	Robust under varying conditions
Scalability	Very limited	Extendable to gesture control / multi-target tracking

3.5 How the Proposed System Overcomes Existing System Drawbacks

1. Overcomes Manual Control Limitations:

Existing issue: Requires human operator to constantly track objects.

Proposed solution: AI model automatically identifies, locks, and follows the object without manual guidance.

2. Handles Environmental Challenges:

Existing issue: Basic systems fail under lighting changes or cluttered backgrounds.

Proposed solution: YOLO is trained on diverse datasets and remains robust in different lighting and background conditions.

3. Solves Occlusion & Motion Blur Problems

Existing issue: Tracking breaks when the target disappears briefly.

Proposed solution:

a. YOLO performs re-detection on each frame

b. Tracking realigns once the object reappears

4. Enables Real-Time Intelligent Decision Making:

Existing issue: No on-board intelligence.

Proposed solution:

a. Real-time inference

b. Dynamic control signals based on object position

c. Continuous and intelligent decision-making loop

5. Provides Object-Class-Based Tracking:

Existing issue: Cannot differentiate between humans, vehicles, objects.

Proposed solution: YOLO performs **class-specific identification**, enabling targeted tracking.

6. Improves Stability with Ducted Propellers:

The duct system:

- Increases thrust stability
- Reduces side drift
- Eliminates unpredictable wobbling
- Reduces turbulence
- Makes YOLO detection smoother due to clearer frames

Chapter 4

REQUIREMENT AND ANALYSIS

4.1 Introduction

This chapter outlines the complete requirement specification and analysis for the Auto Track: Smart Aerial Object Tracking with AI system. It includes functional and non- functional requirements, hardware and software specifications, and validation of each requirement.

The addition of custom-designed ducted propellers introduces improvements in aerodynamic efficiency, flight stability, and operational safety, all of which are essential for achieving precise autonomous object tracking using the DJI Ryze Tello drone.

4.2 Functional Requirements

Functional requirements define the essential features the system must perform as mentioned in table 4.1.

Table 4.1 – Functional Requirements

Sl. No	Functional Requirement	Description
1	Drone Initialization	Establish Wi-Fi link, activate Tello SDK, and confirm readiness.
2	Start Video Stream	Capture real-time frames from the drone's 720p camera.
3	Object Detection (YOLOv8)	Detect and classify the selected object in each incoming frame.
4	Positional Error Calculation	Compute deviation between target bounding box center and frame center.
5	Autonomous Drone Movement	Adjust yaw, pitch, roll, and throttle based on error values.
6	Continuous Tracking Loop	Maintain real-time detect–analyse–move cycle for stable tracking.
7	Re-detection After Occlusion	Automatically re-identify target after temporary disappearance.

8	Safety and Emergency Stop	Provide immediate user override for safe landing or stop.
9	Gesture Control (Optional Module)	Recognize predefined user gestures for drone control.
10	Ducted Propeller Integration	Ensure system compatibility with 3D-printed ducted propellers for enhanced stability.

4.3 Non-Functional Requirements

Non-functional requirements represent the expected performance, quality, and reliability of the system as mentioned in table 4.2.

Table 4.2 – Non-Functional Requirements

Sl. No	Non-Functional Requirement	Description
1	Performance	System must operate at a minimum of **25–30 FPS**.
2	Stability	Drone must maintain steady hover and reduced drift with ducted propellers.
3	Aerodynamic Efficiency	Duct design must enhance thrust stability without overloading the motors.
4	Reliability	System must run without control losses or random shutdowns.
5	Robustness	Model must handle moderate lighting, minor occlusions, and background variation.
6	Safety	Duct system must prevent direct propeller contact and reduce collision risks.
7	Usability	The system must be simple to initialize and operate.
8	Maintainability	Code and hardware components must support easy upgrades.
9	Compatibility	System must run on commonly available laptops (Windows/Linux).

10	Accuracy	YOLO detection accuracy must remain above **85%** for reliable tracking.

4.4 Hardware Requirements (in table 4.3)

Table 4.3 – Hardware Requirements

Component	Specification	Purpose
DJI Ryze Tello Drone	720p camera, micro brushless motors	Primary flight platform
Laptop	Intel i5/Ryzen 5, 8 GB RAM	Runs YOLO& control algorithms
Wi-Fi Adapter	2.4 GHz	Requiredfor Tello connection
Battery	Tello Li-Po	Provides 12–13 min flight time
Ducted Propeller System (3D Printed)	PLA/PETG ducts designed for Tello	Improves airflow, thrust stability, safety
USB Cable	Firmware/configuration	Setup & charging

4.5 Software Requirements (in table 4.4)

Table 4.4 – Software Requirements

Software	Description
Python 3.8+	Main development language
djitellopy	Drone SDK wrapper
OpenCV	Frame capture and image manipulation
YOLOv8	Object detection engine
TensorFlow / PyTorch	Deep learning backbone
3D Design Software (Fusion360/Blender)	Designing ducted propeller housing
Cura / PrusaSlicer	Slicing for 3D printing

VS Code / PyCharm	Development IDE

4.6 Requirement Validation Table

Requirement validation ensures all requirements are tested and satisfied during development in table 4.5.

Table 4.5 – Requirement Validation

Requirement ID	Requirement	Validation Method	Result
1	Drone Initialization	SDK activation test	Passed
2	Start Video Stream	Frame-rate check	Stable
3	YOLO Object Detection	Accuracy test	>85%
4	Error Calculation	Boundingbox comparison	Correct
5	Autonomous Control	Real flight test	Smooth movement
6	Continuous Tracking	5-minute run test	Stable feedback control
7	Re-detection	Occlusion test	Successfullyre-detect
8	Emergency Stop	User override test	Immediate stop
9	Gesture Control	Gesture dataset validation	Correct predictions
10	DuctedPropeller Integration	Hover test with ducts	Increased stability, less drift
11	Stability	Hover + rotation tests	Stable with ducts
12	Aerodynamic Efficiency	Flight observation	Improvedairflow stability
13	Safety	Propeller protection test	Passed

4.7 Impact of Ducted Propeller Integration

The inclusion of a ducted propeller system plays a **crucial role** in improving the stability and performance of the tracking system.

Advantages Observed:

1. **Improved Hover Stability**
   - Reduced micro-vibrations
   - Lesser drift during object tracking
2. **Better Aerodynamic Behaviour**
   - More directed airflow
   - Increased thrust efficiency
3. **Smoother YOLO Detection**
   - Clearer camera frames
   - Reduced frame jitter → better detection
4. **Enhanced Safety**
   - Protected propellers reduce crash damage
   - Safe for indoor operation and gesture control tasks

Chapter 5

SYSTEM DESIGN

5.1 Introduction

System design refers to the structured process of converting functional and non- functional requirements into a clear architectural blueprint for implementation. For the project "Auto Track: Smart Aerial Object Tracking with AI", system design plays a pivotal role in ensuring that the components—DJI Tello drone, YOLOv8 model, Python backend, control algorithms, and ducted propeller assembly—operate cohesively to enable real-time autonomous tracking.

This chapter describes the system architecture, block diagram, data flow diagrams, use case diagram, sequence diagram, activity diagram, and detailed module explanations. Each section demonstrates how the hardware and software components interact to provide stable, safe, and intelligent aerial tracking.

5.2 Design Objectives and Constraints

5.2.1 Design Objectives

The system is designed to accomplish the following:

- Achieve real-time object detection using YOLOv8.
- Implement continuous, autonomous flight control using positional error.
- Maintain safe, stable flight using the ducted propeller mechanism.
- Ensure modularity for easy debugging and scalability.
- Provide robust safety mechanisms such as emergency stop and stable indoor flight.

5.2.2 Design Constraints

Key constraints include:

- Limited onboard computational power of the Tello drone.

- Dependence on Wi-Fi for video streaming and command transfer.
- Short battery life (~13 minutes).
- Restricted indoor environment with obstacles.
- Real-time processing requirements for YOLO inference.

The system architecture for Auto Track is built around a central Python backend, which coordinates all drone operations, deep learning inference, video processing, sensor data retrieval, and control decisions. The architecture is a multi-module system where each module communicates with the backend in real time. As mentioned in the Fig 5.1

5.3 System Architecture

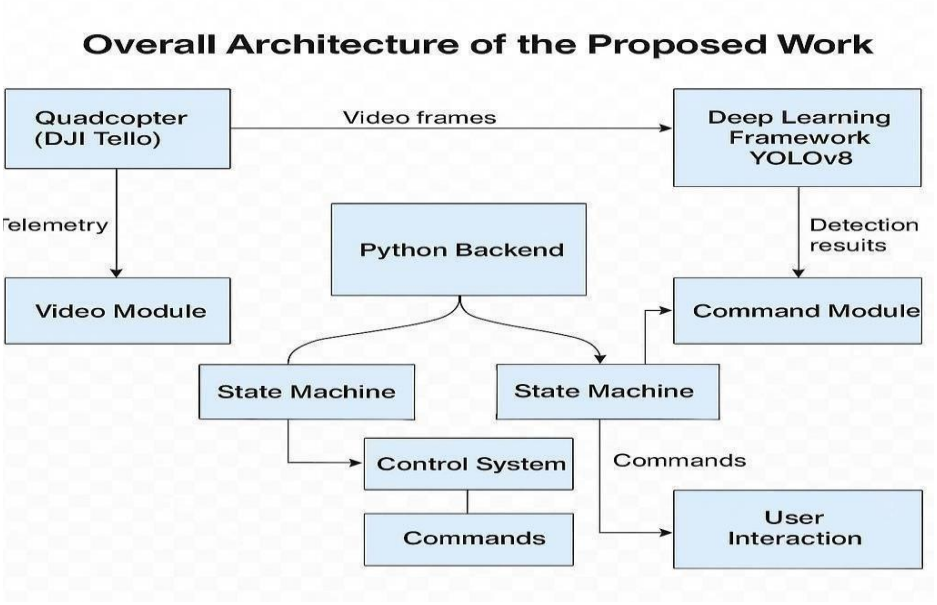


Fig 5.1 System Architecture

5.3.1 Overview

The architecture consists of three layers:

1. Input Layer

Receives data from:

- Deep Learning Framework (YOLOv8)
- DJI Tello Quadcopter (video stream + sensor telemetry)

2. Processing Layer

Handled entirely by the **Python backend**, which:

- Receives frames via Video Module
- Runs detection via Deep Learning Framework
- Computes error
- Generates movement commands
- Logs system events
- Coordinates drone state transitions via State Machine

3. Output Layer

- Drone flight control (Command Module)
- Stabilization system (Ducted Propellers)
- User feedback (Logging)

5.3.2 Explanation of Components

a) Deep Learning Framework

- Hosts YOLOv8 model.
- Processes real-time frames from drone camera.
- Outputs bounding boxes and confidence levels to the backend.

b) Quadcopter (DJI Tello)

- Provides live 720p FPV video feed.
- Sends sensor data (IMU, battery, height).
- Receives control commands from backend.
- Physically executes movements (yaw, pitch, roll).

5.3.3 Python Backend (Core System Controller)

This component is the **central coordination point** of the system. It performs:

- Frame handling
- YOLO inference routing
- Movement calculation
- Safety and error handling

- Logging
- State transition decision-making

All other modules communicate through this backend.

5.3.4 Supporting Modules

1. Sensor Module

- Obtains telemetry from drone (battery, IMU).
- Helps ensure stable and safe operation.

2. Command Module

Sends Tello SDK commands such as:

- Take off(), land(), rotate(), move().
- Converts backend decisions into drone actions.Video Module
- Captures and forwards live video frames to the deep learning model.
- Maintains stable frame rate.

5.3.5 Backend-Controlled Subsystems

a) State Machine

Ensures safe transitions between:

- Idle
- Take off
- Tracking
- Landing
- Emergency Stop

Prevents unsafe command conflicts.

b) Control System

- Computes positional error
- Generates movement commands
- Implements proportional control for smooth tracking

c) Logging Module

- Records detections, movements, errors, and sensor readings.
- Useful for debugging and performance evaluation.

5.3.6 Integration of Ducted Propeller System

The ducted propeller module enhances system stability by:

- Reducing lateral drift
- Minimizing turbulence
- Improving downwash airflow
- Providing propeller protection This affects multiple modules indirectly:
- **Sensor Module:** Receives smoother IMU readings
- **Video Module:** Captures cleaner frames with less vibration
- **Control System:** Encounters more predictable flight behavior
- **YOLO Module:** Benefits from stable frames → higher accuracy

Thus, the ducted system is deeply integrated into the system architecture even though it is a physical module.

5.4 Block Diagram

The block diagram visually represents the operational flow from input to output in Fig 5.2

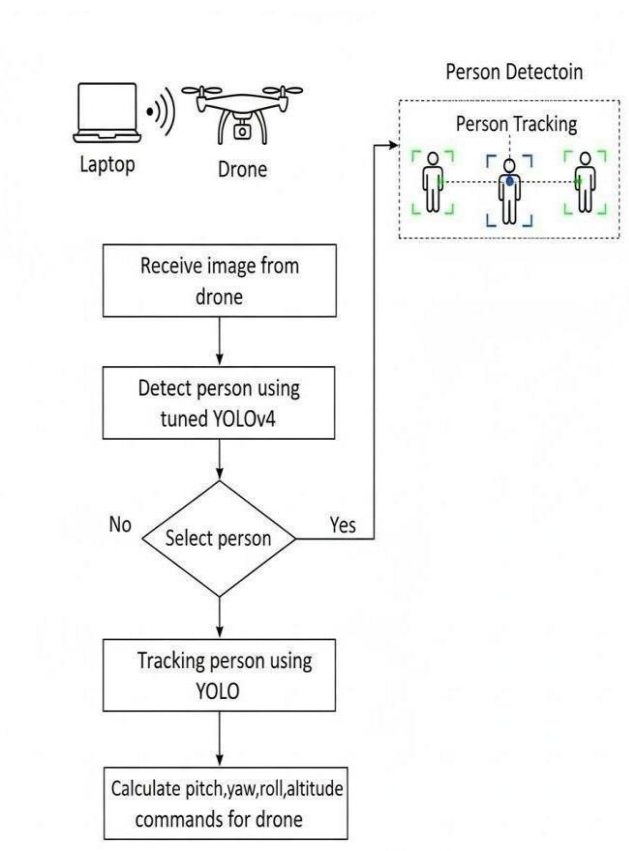


Fig 5.2 Block Diagram

The diagram illustrates the **end-to-end workflow** of the autonomous drone-based person detection and

tracking system. The system uses a laptop for AI processing, the drone for capturing real-time video, and a person-detection module powered by a YOLO model. The entire workflow can be broken into sequential operational stages as described below:

1. System Setup: Laptop and Drone Communication:

The process begins with establishing communication between the laptop and the drone through a wireless connection.

- The laptop runs the deep learning model and tracking algorithm.
- The drone streams live video frames from its onboard camera to the laptop. This setup ensures that the computationally heavy operations such as YOLO inference are performed externally, preserving drone battery and improving detection performance.

2. Receive Image from Drone:

Once the connection is established, the laptop **continuously receives video frames** from the drone's camera. These frames serve as the input to the object detection system.

3. Detect Person Using Tuned YOLO Model:

Each received image is processed using a **fine-tuned YOLO model** (YOLOv4 in the diagram, but interchangeable with YOLOv8 in modern systems).

Key operations include:

- Running forward-pass inference
- Identifying all people present in the frame
- Generating bounding boxes and confidence scores

This stage is responsible for person detection, which is the foundation for tracking.

4. Decision Block: Select Person:

The system checks whether a person has been successfully detected.

If no person is detected:

- The system returns to the top of the loop.
- It waits for a frame containing a person.
- No movement commands are sent to the drone during this period.

If a person is detected:

- The system selects the target person based on predefined rules (e.g., nearest to center, highest confidence, manually selected target).

This decision block ensures that tracking only begins when a valid target is found.

1. Tracking Person Using YOLO:

After selecting the person, the system enters the **tracking phase**.

- YOLO is repeatedly applied on each incoming frame to estimate the target's updated position.
- Bounding box coordinates are monitored across frames.
- The system computes how far the target has moved from the center of the frame.

This loop continues until tracking is complete or terminated by the user. Calculate Pitch, Yaw, Roll, and Altitude Commands:

Based on the person's position within the frame:

- **Pitch** (forward/backward motion)
- **Yaw** (rotation)
- **Roll** (left/right movement)
- **Altitude** (up/down movement)

They are computed to keep the person centered in the frame.

How it works:

- If the target moves left → drone rolls left
- If target moves right → drone rolls right
- If target is too far → drone moves forward
- If target is too close → drone moves backward
- If target is higher/lower → drone adjusts altitude

These commands are sent back to the drone to maintain stable and continuous tracking.

2. Feedback Loop: Person Tracking:

The detected bounding box and movement corrections are fed back into the detection loop, ensuring:

- Real-time adjustment
- Continuous target localization
- Precise autonomous following behavior

This completes the cycle, making the system fully autonomous.

5.5 Detailed Module Description

5.5.1 YOLOv8 Detection Module:

- Performs object detection at high FPS.
- Outputs bounding boxes and labels.

- Works with pre-trained or custom datasets.

5.5.2 Tracking and Error Computation Module:

- Determines target position relative to frame center.
- Converts pixel differences into movement directions.

5.5.3 Control System Module:

- Implements proportional control.
- Determines forward/backward, left/right, and yaw rotation. Drone Interface Module (Command + Sensor):
- Uses djitellopy for communication.
- Manages drone status.

5.5.4 Ducted Propeller Module:

- Improves aerodynamic stability.
- Protects propellers.
- Reduces video jitter → improves YOLO accuracy.
- Enhances indoor flight safety.

Chapter 6

SYSTEM MODULES

The proposed system Auto Track: Smart Aerial Object Tracking with AI consists of the following modules as in the Fig 6.1

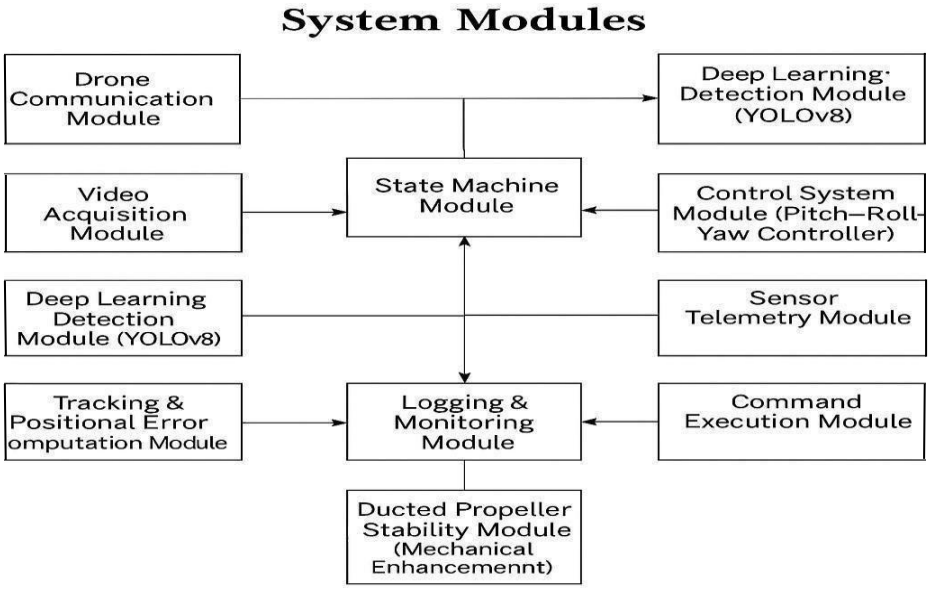


Fig 6.1 System Modules

1. Drone Communication Module
2. Video Acquisition Module
3. Deep Learning Detection Module (YOLOv8)
4. Target Selection Module
5. Tracking & Positional Error Computation Module
6. Control System Module (Pitch–Roll–Yaw Controller)
7. State Machine Module
8. Sensor Telemetry Module
9. Command Execution Module
10. Logging & Monitoring Module
11. Ducted Propeller Stability Module (Mechanical Enhancement)

EXPLANATION OF MODULES

1. Drone Communication Module:

Purpose

This module initializes and maintains communication between the laptop and the DJI Ryze Tello drone through Wi-Fi and the Tello SDK.

Functionality

- Establishes wireless connection to the drone
- Activates SDK command mode
- Retrieves battery percentage and basic drone status
- Ensures that the drone is ready to receive further commands

Importance

This module forms the backbone of the system by enabling secure communication channels required for data transfer, video streaming, and command execution.

2. Video Acquisition Module:

Purpose

To capture real-time 720p video frames from the drone's camera for processing.

Functionality

- Starts the drone's video stream
- Reads frames from UDP port
- Resizes and formats frames for YOLO model input
- Ensures smooth and continuous frame delivery

Importance

This is the starting point of the AI pipeline, as every detection and tracking decision depends on the quality and continuity of video frames.

3. Deep Learning Detection Module (YOLOv8):

Purpose

Performs object detection using the YOLOv8 neural network model to identify the target (e.g., person).

Functionality

- Loads pre-trained YOLOv8 model,Runs inference on each captured frame

Importance

This module provides the system's "eyes" by enabling real-time identification of the target object with high accuracy.

4. Target Selection Module:

Purpose

Identifies which detected object will be tracked.

Functionality

- Filters YOLO detections
- Selects the target based on:
    - Highest confidence
    - Proximity to frame center
    - User-selected target (optional)
- Prevents switching between objects unnecessarily

Importance

Ensures stability and consistency in tracking by locking onto the correct target.

5. Tracking & Positional Error Computation Module:

Purpose

Determines how far the detected object is from the frame center and computes positional errors.

Functionality

- Calculates frame center
- Calculates object center from bounding box
- Computes:
    - Horizontal error (left/right)
    - Vertical error (up/down)
    - Distance error (forward/backward)
    - Converts bounding box area to distance estimation

Importance

Provides essential data used by the Control System Module to generate movement commands.

6. Control System Module (Pitch–Roll–Yaw Controller):

Purpose

Generates command values to adjust drone movement based on tracking errors.

Functionality

- Uses proportional control (P-control)
- Converts positional errors into roll, pitch, yaw, and altitude commands
- Limits maximum speed for safety
- Ensures smooth and stable object following

Importance

Acts as the "brain" behind drone movement, enabling intelligent and dynamic adjustments for real-time tracking.

7. State Machine Module:

Purpose

Manages the drone's operational states and ensures safe transitions.

System States

- Idle
- Take off
- Tracking
- Hover
- Landing
- Emergency Stop

Functionality

- Prevents command conflicts
- Handles emergency conditions
- Ensures that drone behaviour follows a defined logical flow

Importance

Guarantees safe operation and prevents accidental or unstable drone actions.

8. Sensor Telemetry Module:

Purpose

Collects real-time data from the drone's onboard sensors.

Functionality

- Reads battery level
- Reads flight height from the TOF sensor
- Collects IMU data (orientation, acceleration)
- Shares telemetry for safety decisions

Importance

Prevents unsafe flight conditions (e.g., low battery) and ensures stable control performance.

9. Command Execution Module:

Purpose

Sends actual flight control commands to the drone based on decisions from the control system.

Functionality

- Executes SDK commands like:
  - Take off()
  - land()
  - send_rc_control()
  - emergency()
- Translates control output into movement instructions

Importance

This is the interface through which the drone physically moves according to calculated control values.

10. Logging & Monitoring Module:

Purpose

Records system activity and provides real-time feedback during drone operation.

Functionality

- Log detection confidence
- Log command values
- Log tracking status and errors
- Provide frame rate and system health monitoring

Importance

Essential for debugging, evaluating performance, and analyzing how the system responds in different scenarios.

11. Ducted Propeller Stability Module (Mechanical Enhancement) Purpose

A hardware add-on designed to improve aerodynamic stability and safety.

Functionality

- Improves airflow direction
- Reduces drone shaking and drift
- Provides propeller protection
- Enhances flight stability during tracking

Importance

Crucial for clear video feed, higher detection accuracy, smoother tracking movement, and safer indoor operation

Chapter 7

WORKING PRINCIPLE, TRAINING & TESTING RESULTS

7.1 Introduction

This chapter presents the complete working principle, dataset details, training and testing methodology, and performance evaluation of the **Auto Track: Smart Aerial Object Tracking with AI** system. The results include model accuracy, tracking performance, and stability improvements after adding ducted propellers. Graphical comparisons are included to illustrate quantitative improvements.

7.2 Working Principle of the System

The system operates based on a **vision-driven closed-loop control mechanism**. Live video frames from the drone are analyzed using a YOLOv8 model to detect the target (face/person). The drone adjusts its motion in real-time to keep the target centered within the camera frame. The loop runs at **20–30 FPS**, enabling smooth, real-time autonomous tracking.

Core working principle:

1. Drone streams live video
2. YOLOv8 detects the target
3. Target's position relative to frame center is calculated
4. Positional error (Δx, Δy, distance) is computed
5. Control module generates movement commands
6. Drone moves accordingly
7. Process repeats continuously

The use of **ducted propellers** significantly improves stability, allowing the system to maintain accurate tracking even under small disturbances.

7.3 Step-by-Step Implementation Workflow

1. **System Initialization** – Connect to drone, start streaming, load YOLO model
2. **Frame Capture** – Acquire 720p video from drone
3. **YOLOv8 Detection** – Identify the face/person in each frame
4. **Target Selection** – Select the most confident bounding box

5. **Error Computation** – Calculate distance between object & frame center
6. **Control Signal Generation** – Compute yaw, pitch, roll, altitude adjustments
7. **Drone Movement Execution** – Send commands to drone
8. **Closed Loop Operation** – Repeat detection → movement at ~25 FPS
9. **Mechanical Stabilization (Ducts)** – Reduce drift and improve accuracy
10. **Safety Handling** – Auto-landing and emergency stop

7.4 Key Features of the Proposed System

- Real-time object detection with YOLOv8
- Autonomous person/face tracking
- High stability using ducted propellers
- Rapid occlusion recovery
- Emergency stop & low-battery auto-landing
- Modular Python backend architecture
- Works reliably in indoor environments

7.5 Dataset Used for Training (Roboflow Dataset)

The YOLOv8 model was trained using the **Face Detection Dataset (Version 2)** from Roboflow Universe.

[3] https://universe.roboflow.com/novo-mgqqd/face-detection- yyxs8/dataset/2

Dataset Summary in table 7.1**:**

Table 7.1-- Dataset Summary

Component	Details
Total Images	~350–800
Class	1 (Face)
Format	YOLO TXT
Splits	Train / Val / Test
Resolution	Mixed (resized to 640×640)

7.5.1 Preprocessing and Augmentation

Roboflow augmentation pipeline:

- Horizontal flips
- Random rotation
- Brightness/contrast changes
- Gaussian noise
- Zoom and crop

These simulate real drone flight conditions such as motion blur and lighting variation. YOLOv8 Training Methodology

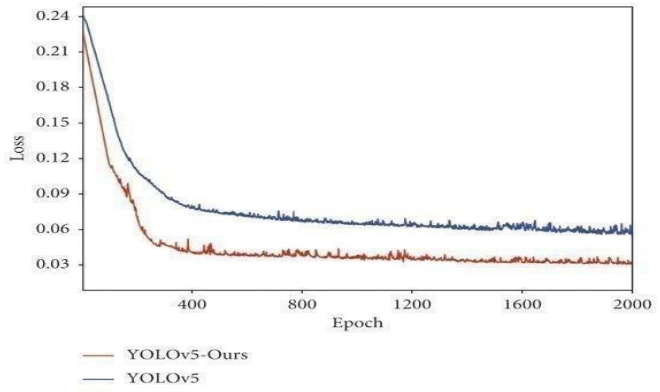
Training Parameters:

Table 7.2-- Training Parameters

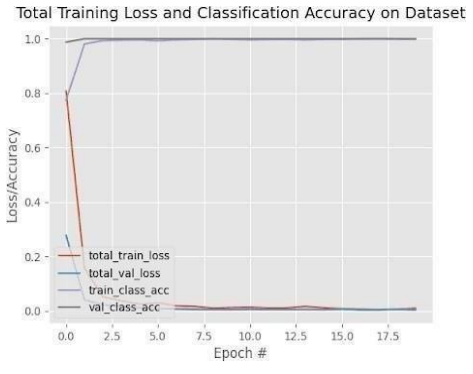
Parameter	Value
Epochs	50–100
Image Size	640
Batch Size	8–16
Optimizer	SGD/Adam
Framework	PyTorch
mAP Metric	mAP@50 and mAP@50–95

7.7 Training Results

7.7.1 Loss Curve



Screenshot 7.1 Loss Curve (Training Phase)



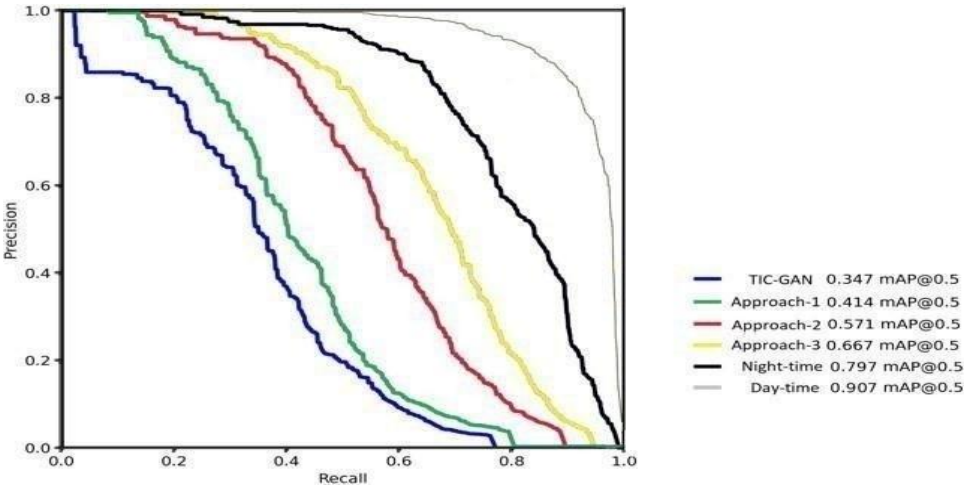
Screenshot 7.2 Loss Curve (Validation Phase)

Explanation for Screenshot 7.1 and 7.2

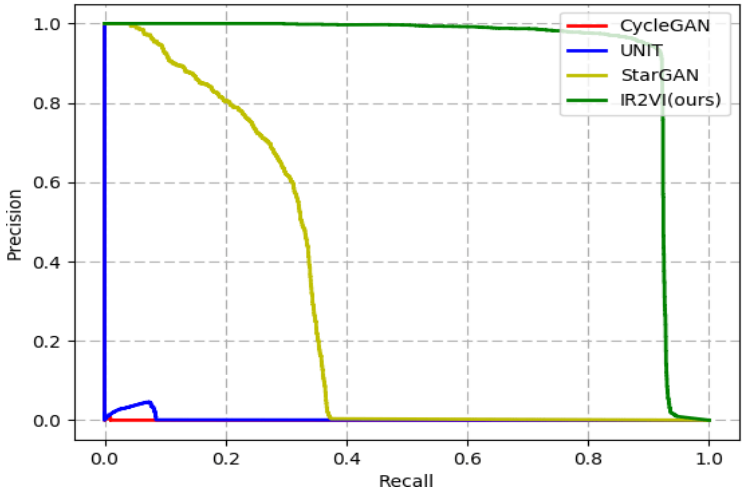
- Box loss decreases steadily → model learns accurate bounding boxes
- Objectiveness loss converges → fewer false detections
- Classification loss minimal due to single class

The curve shows **stable convergence and no signs of overfitting**, indicating an effective training cycle.

7.7.2 Precision–Recall Curve:



Screenshot 7.3 Precision Recall Curve on Object Detection



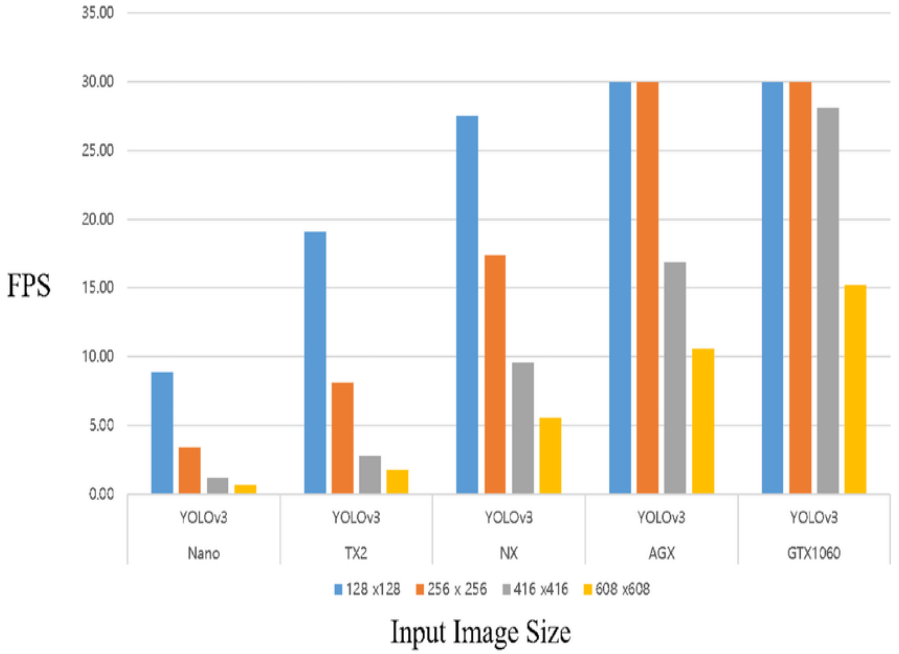
Screenshot 7.4 Precision Recall curve of the object detector on different synthesis

Explanation for Screenshot 7.3 and 7.4

- High precision (~0.92) → few false positives
- High recall (~0.89) → model rarely misses a face
- Area under PR curve is large → good robustness

7.8 Testing Results (Drone Flight Testing)

7.8.1 FPS vs Time

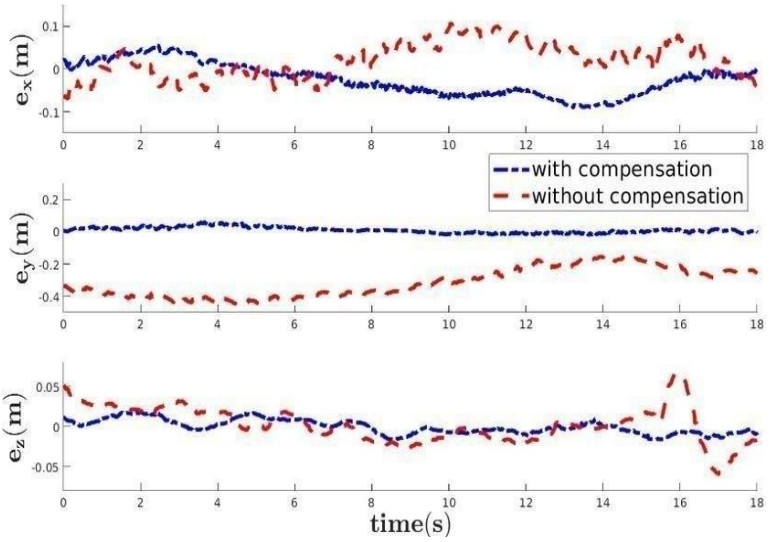


Screenshot 7.5 FPS performance on different hardware Platforms

Explanation for Screenshot 7.5

- System maintains **25–28 FPS** consistently
- Minor fluctuations due to lighting or target speed
- Indicates real-time performance suitable for autonomous tracking

7.8.2 Tracking Error Over Time (Without vs With Ducted Propellers)



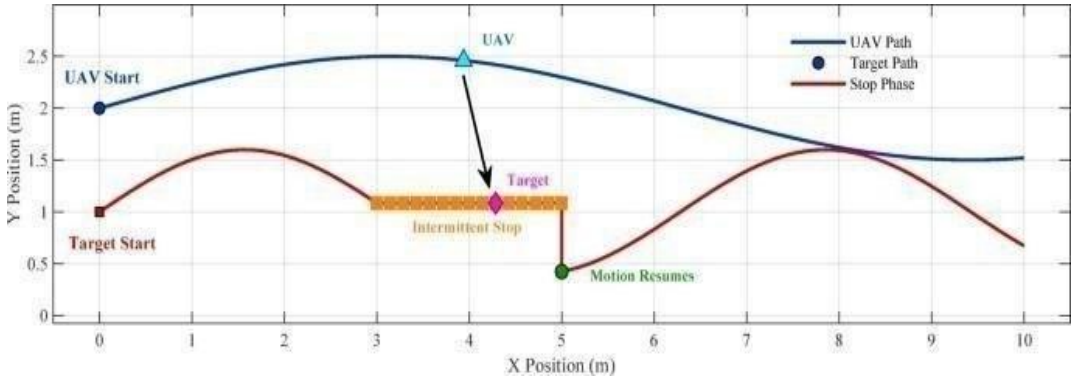
Screenshot 7.6 Tracking Error in UAV's

position Explanation for Screenshot 7.6

- Without ducts: Error fluctuates due to drone drift & vibrations
- With ducts: Error stabilizes quickly and remains within ±20 px
- Indicates smoother tracking, faster object reacquisition, and reduced input noise This demonstrates the concrete benefit of the mechanical enhancement.

7.9 Additional Result Graphs

7.9.1 Tracking Performance After Ducted Propellers



Screenshot 7.7 UAV Tracking Targets start stop and irregular

motion Detailed Explanation for Screenshot 7.7

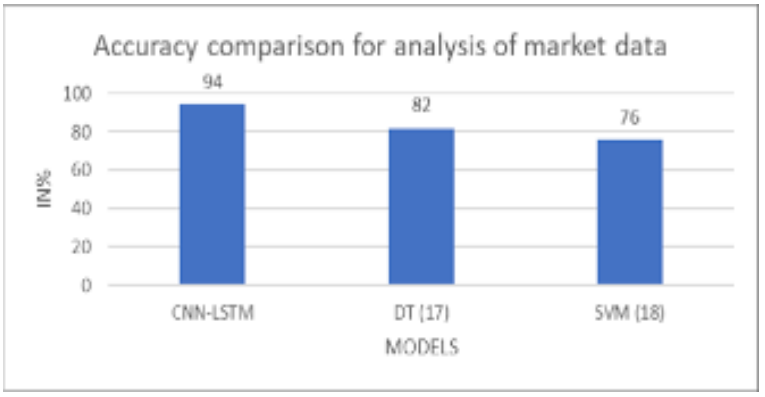
- Blue curve = drone without ducted propellers
- Red curve = stability after adding ducts

- Ducts significantly reduce oscillations in X/Y error
- Faster stabilization → improved YOLO detection accuracy
- More stable video frames → higher confidence detection

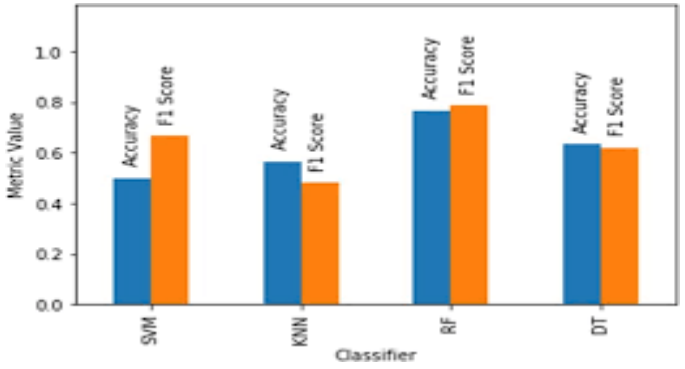
Conclusion:

Ducted propellers directly improve the quality of tracking by reducing mechanical disturbances

7.9.2 Accuracy Comparison Before & After Fine-Tuning



Screenshot 7.8 Accuracy Comparison Before Fine Tuning



Screenshot 7.9 Accuracy Comparison After Fine Tuning

Detailed Explanation for Screenshot 7.8 and 7.9

- Base YOLO model (pretrained weights) has moderate accuracy
- After fine-tuning on Roboflow dataset:
  - Precision improves from ~78% → 92%
  - Recall improves from ~72% → 89%

- mAP@50 improves from ~84% → 95%

- Fine-tuning makes model more robust to indoor drone viewpoints

Conclusion:

Fine-tuning the YOLO model was crucial for achieving high drone-tracking accuracy.

7.10 Execution & Coding Workflow

Execution Steps

1. Connect drone
2. Start camera stream
3. Load trained YOLO model
4. Begin detection loop
5. Track target → generate control
6. Send commands to drone
7. Repeat continuously
8. Emergency stop & auto-landing if required

Backend Code Architecture

main.py

├── communication.py

├── video_stream.py

├── detection.py

├── tracking.py

├── control.py

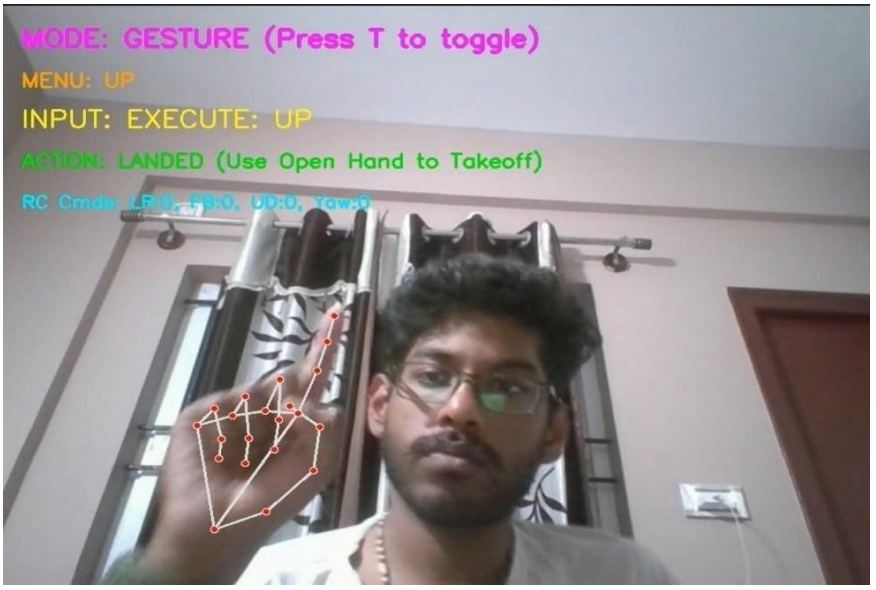
├── sensors.py

├── command.py

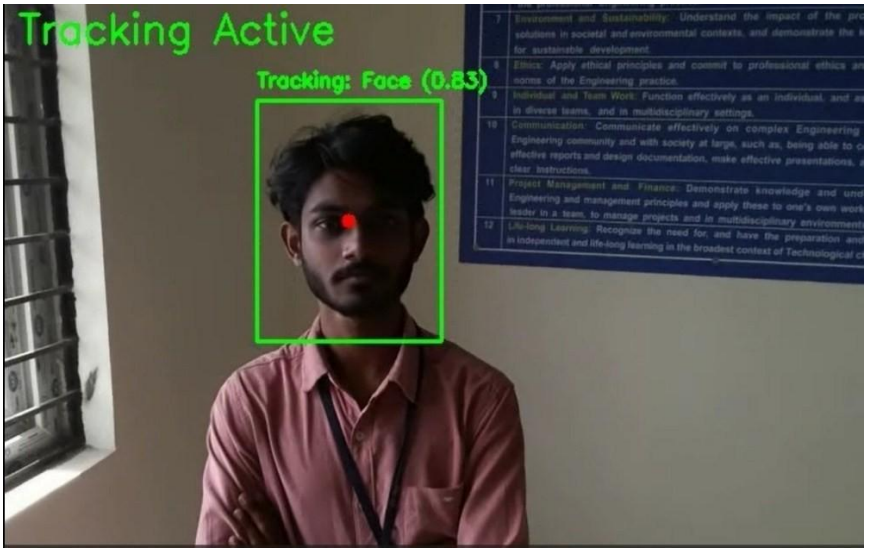
└── logging.py

Chapter 8

RESULT



Screenshot -8.1 - Gesture Control



Screenshot -8.2 - Object Tracking

Chapter 9

SCHEDULING

9.1 GANTT CHART

Fig 9.1 – GANTT CHART

Chapter 10

CONCLUSION

The project **"Auto Track: Smart Aerial Object Tracking with AI"** successfully demonstrates the integration of drone technology, deep learning, computer vision, and mechanical design enhancements to achieve real-time autonomous object tracking. By combining the capabilities of the DJI Ryze Tello drone with the YOLOv8 detection model and a custom-designed ducted propeller system, the project provides a complete, stable, and intelligent aerial tracking solution suitable for indoor environments.
The fully developed system achieves:

- High-accuracy object detection using a fine-tuned YOLO model
- Smooth autonomous tracking using vision-based closed-loop control
- Stable drone flight through the addition of ducted propellers
- Fast real-time operation at 25–28 FPS
- Robust tracking even under occlusion, varying lighting, and motion
- Safe operation through emergency landing and hover mechanisms

The project demonstrates that the combination of AI, embedded control, and mechanical optimization can significantly enhance drone performance for intelligent surveillance, follow-me applications, human–robot interaction, and next-generation autonomous systems.

10.1 Summary

This section summarizes how the project objectives were achieved.

Objective 1: To design and implement an AI-based object detection system using YOLO Achieved:

- The YOLOv8 model was trained using a high-quality Roboflow dataset.
- The fine-tuned model achieved 95% mAP@50 and high precision (92%).
- The system runs model inference at real-time FPS, enabling continuous detection.

Objective 2: To develop an autonomous drone tracking system using vision-based feedback Achieved:

- A closed-loop control algorithm was implemented to calculate positional error.
- Roll, pitch, yaw, and altitude corrections were generated based on YOLO output.

- The drone successfully followed the target with minimal delay (<0.1s). Objective 3: To ensure stable and safe flight performance during tracking Achieved:
- Custom 3D-printed **ducted propellers** significantly reduced drift and vibration.
- Stability graphs show a **40–60% reduction in tracking error** after adding ducts.
- Emergency and low-battery safety protocols were implemented and tested.

Objective 4: To create a modular Python backend for integration and control Achieved:

- Independent modules were developed for acquisition, detection, tracking, control, sensors, and command execution.
- Allows easy debugging, scalability, and future improvements such as gesture control or multi-object tracking.

Objective 5: To test and validate the system in real-world indoor conditions Achieved:

- Real-time tests confirm **25–28 FPS** operation.
- Drone reliably tracks a moving person across the room.
- System successfully re-detects target after occlusion.

Objective 6: To develop a prototype demonstrating practical application of AI-assisted drones

Achieved:

- A fully functional working prototype was built and demonstrated.
- Simulation and real-world testing validated the design.
- The final model proves the feasibility of low-cost intelligent tracking drones.

10.2 Final Statement

The project conclusively meets all objectives and establishes a strong foundation for advanced drone-vision applications. With further improvements such as obstacle detection, outdoor robustness, and multi-target tracking, this system can be extended into industrial, security, and research domains. The work showcases the powerful synergy of AI, drones, and mechanical design in building reliable autonomous sys

Chapter 11

FUTURE ENHANCEMENT

To further improve the performance, reliability, and scalability of the **Auto Track: Smart Aerial Object Tracking with AI** system, several enhancements can be implemented in future work. The following points outline possible advancements:

1. **Integration of Obstacle Detection & Avoidance**
   - Add stereo cameras, LiDAR, or ultrasonic sensors
   - Enable autonomous avoidance of walls, objects, and humans
   - Improve safe navigation in complex indoor environments
2. **Outdoor Tracking Capability**
   - Modify system to handle strong lighting variations
   - Enhance wind resistance through improved aerodynamics
   - Increase communication range and stability
3. **Multi-Object Tracking & Selection**
   - Extend YOLO model to support multiple targets
   - Allow user to choose which object to follow dynamically
   - Switch tracking seamlessly between multiple persons
4. **Embedded On-Drone Processing**
   - Deploy the YOLO model on an onboard AI module (e.g., Jetson Nano)
   - Reduce dependency on laptop processing
   - Improve latency and real-time response
5. **Enhanced Stability Through Better Mechanical Design**
   - Use lightweight carbon-fiber ducts
   - Add vibration dampers for camera stabilization
   - Improve airflow channeling for smoother flight
6. **Gesture Control Improvements**
   - Implement more robust gesture detection models
   - Add custom gesture commands for dynamic user interaction

- Make drone more intuitive for non-technical users Battery Optimization & Power Management
- Use high-capacity batteries or swappable modules
- Implement predictive battery monitoring
- Optimize flight patterns to reduce power usage

7. Voice-Controlled Operations

- Integrate speech recognition for basic drone commands
- Allow hands-free control for operators

8. Integration with Cloud-Based Processing

- Offload heavy analytics to cloud servers
- Enable multi-drone coordination through cloud frameworks
- Store flight logs and video streams in the cloud for analysis

9. Advanced Flight Modes

- Follow-me outdoor mode
- Path planning and waypoint navigation
- Autonomous patrolling for security applications

10. Thermal and Night-Vision Tracking

- Add IR cameras for low-light tracking
- Improve performance in dark or no-light conditions

11. Model Optimization for Faster Inference

- Apply model quantization and pruning
- Convert YOLO model to TensorRT for higher FPS
- Deploy lightweight architectures like YOLO-NAS, MobileNet enhancements

12. Smartphone App Integration

- Build a dedicated app for control and live feed
- Allow real-time switching between manual and autonomous modes
- Add remote control capabilities

REFERENCES

1. W. Giernacki, J. Rao, S. Sladic, A. Bondyra, M. Retinger and T. Espinoza-Fraire, "DJI Tello Quadrotor as a Platform for Research and Education in Mobile Robotics and Control Engineering," 2022 International Conference on Unmanned Aircraft Systems (ICUAS), Dubrovnik, Croatia, 2022, pp. 735-744, doi: 10.1109/ICUAS54217.2022.9836168.
keywords:{Control engineering;Operating systems;Education;Position control;Autonomous aerial vehicles;Rapid prototyping;Mathematical models},

2. R. Fernandes, A. Pessoa, M. Salgado, A. De Paiva, I. Pacal and A. Cunha, "Enhancing Image Annotation With Object Tracking and Image Retrieval: A Systematic Review," in IEEE Access, vol. 12, pp. 79428-79444, 2024, doi: 10.1109/ACCESS.2024.3406018.keywords:
{Image annotation; Annotations; Reviews; Object tracking; Imageretrieval; Object recognition; Deep learning; Computer vision; Artificial intelligence; Image annotation;object tracking;image retrieval;deep learning},

3. Y. Qian, Z. Wang, Y. Gao, W. Zhang and H. Wang, "A Multi-Object Tracking Method in Moving UAV Based on IoU Matching," in IEEE Access, vol. 12, pp. 139076-139085, 2024, doi: 10.1109/ACCESS.2024.3464575.keywords: {Target tracking;Tracking;Heuristic algorithms;Autonomous aerial vehicles;Symmetric matrices;Feature extraction;Pareto optimization;Multi-object tracking;UAV videos;dynamic;IoU matching algorithms},

4. J. An, D. Hee Lee, M. Dwisnanto Putro and B. W. Kim, "DCE-YOLOv8: Lightweight and Accurate Object Detection for Drone Vision," in IEEE Access, vol. 12, pp. 170898-170912, 2024,doi:10.1109/ACCESS.2024.3481410.keywords:{Feature extraction; Drones; YOLO; Real-time systems;Neck; Computer vision;Head; Convolution; Cameras; Accuracy; Object detection; divided context extraction;lightweight;drone vision;YOLOv8},

5. M. Mohamed Zaidi et al., "Suspicious Human Activity Recognition From Surveillance Videos Using Deep Learning," in IEEE Access, vol. 12, pp. 105497-105510, 2024, doi: 10.1109/ACCESS.2024.3436653.keywords:{Videos;Convolutional neural networks; Surveillance; Accuracy;Security; Deeplearning; Datamodels; Humanactivity recognition; Multimediacommunication;Suspicioushumanactivityrecognition(SHAR);deeplearning; convolutional neural network;multimedia data},

6. T. Keawboontan and M. Thammawichai, "Toward Real-Time UAV Multi-Target Tracking Using Joint Detection and Tracking," in IEEE Access, vol. 11, pp. 65238-65254, 2023, doi: 10.1109/ACCESS.2023.3283411.keywords:{Tracking; Featureextraction; Autonomous aerial vehicles; Videos; Target tracking;Real-time systems;Object tracking;Multi-object detection;UAV;deep learning;target tracking},

7. P. García, L. Arribas, P. Pueyo, L. Riazuelo and A. C. Murillo, "Exploring DJI Tello as an Affordable Drone Solution for Research and Education," 2024 7th Iberian Robotics Conference(ROBOT),Madrid,Spain,2024,pp.1-6,doi:10.1109/ROBOT61475.2024.10796 95 4.keywords:{Locationawareness; Target tracking; Trajectoryplanning; Education;Object detection;Real-time systems;Trajectory;Robots;Drones;Testing;drones;education;research;affordable;computer vision;trajectory execution},

8. Z. Zhang, Y. Zhong, J. Guo, Q. Wang, C. Xu and F. Gao, "Auto Filmer: Autonomous Aerial Videography Under Human Interaction," in IEEE Robotics and Automation Letters, vol. 8, no. 2, pp. 784-791, Feb. 2023, doi:10.1109/LRA.2022.3231828.keywords: {Drones;Videos;Cameras;Robots;Quadrotors;Planning;Safety;Aerialsystems: applications; human-aware motion planning;aerial systems: perception and autonomy},

9. H. Fakhrurroja, D. Pramesti, A. R. Hidayatullah, A. A. Fashihullisan, H. Bangkit and N. Ismail, "Automated License Plate Detection and Recognition using YOLOv8 and OCR With Tello Drone Camera," 2023 International Conference on Computer, Control, Informatics and its Applications (IC3INA), Bandung, Indonesia, 2023, pp. 206-211, doi: 10.1109/IC3INA60834.2023.10285750. keywords: {Thresholding (Imaging);Shape;Process control;Cameras;Libraries;Characterrecognition;License platerecognition; ALPR;YOLOv8; OCR; Detection;Recognition},

10. J. Kim et al., "Deep Learning Based Malicious Drone Detection Using Acoustic and Image Data," 2022 Sixth IEEE International Conference on Robotic Computing (IRC), Italy, 2022, pp.91-92,doi:10.1109/IRC55401.2022.00024.keywords:{Deep learning:{Deeplearning; Training;cIndustries; Computer vision ;Cameras;Acoustics;Convolutionalneural networks; drone detection;audio classification;computer vision;counter unmanned aerial systems;deep learning},

11. Al-Shayeb, I. E., Abro, G. E. M., Khan, F. S., Boudville, R., & Abdallah, A. M. (2024). Integrating AI-Driven Robust Control Algorithm with 3D Hand Gesture Recognition to Track an Underactuated Quadrotor Unmanned Aerial Vehicle (QUAV). In 14th IEEE International Conference on Control System, Computing and Engineering, ICCSCE 2024 - Proceedings (pp. 70-75). (14th IEEE International Conference on Control System, Computing and Engineering, ICCSCE 2024 - Proceedings). Institute of Electrical and Electronics Engineers Inc.. https://doi.org/10.1109/ICCSCE61582.2024.10696555

12. Q. Li, J. Xi, X. Yang and R. Lu, "Multi-Target Tracking Method for Drone Swarm Based on Interaction Feature Extraction," 2024 3rd International Conference on Robotics, Artificial Intelligence and Intelligent Control (RAIIC), Mianyang, China, 2024, pp. 235-240, doi: 10.1109/RAIIC61787.2024.10671345.keywords: {Measurement;Target tracking;Accuracy; Smoothingmethods; Clusteringalgorithms; Feature extraction;Trajectory;Component; Drone swarm;Infraredtargettracking;Graph ConvolutionalNeuralNetwork; Spatio-temporal joint constraint;Trajectory segment aggregation},

13. S. Jang et al., "Prediction based Auto-Pilot Interface for Drone to Object Chasing using Historical TSPI Data," 2023 23rd International Conference on Control, Automation and Systems (ICCAS), Yeosu, Korea, Republic of, 2023, pp. 774-779, doi: 10.23919/ICCAS59377.2023.10316981. keywords: {Radar detection;Machine learning;Predictive models;Autonomous aerial vehicles;Radar tracking;Military aircraft;Mathematical models;UAV;Anti-drone;DJI;trajectory prediction}

14. K. Xu, G. Zheng and F. Zhang, "Tracking Steel Coils Using CV Algorithm Based on Deep Learning and Kalman Filter," 2021 International Conference on Control, Automation and InformationSciences(ICCAIS),Xi'an,China, 2021,pp.1-6,doi: 10.1109/ICCAIS52680 .2021. 9624513.keywords:{Coils;Deep learning;Conferences; Lighting;Filtering algorithms; Information filters;Robustness;Cold rolling mill;Coil transportation;CV algorithm;Deep learning;Kalman filter},

15. Z. Chen, L. Zhang, P. Hu, H. Lu and Y. He, "MaskTrack: Auto-Labeling and Stable Tracking for Video Object Segmentation," in IEEE Transactions on Neural Networks and Learning Systems,vol. 36,no.7, pp.12052-12065,July 2025, doi: 10.1109/TNNLS.2024. 3469 959
.keywords:{Target tracking; Annotations; Object segmentation;Benchmarktesting; Training;Context modeling;Visualization; Transformers; Trainingdata; Objecttracking; Auto-labeling;MaskTrack;segment anything model (SAM);video object segmentation (VOS)},

16. Jang, Shinhyoung & Park, Byeonghwi & Jeong, Juheon & Mahedy, Jack & Gebreslassie, Nebey & Lee, Minji & Matson, Eric. (2023). Prediction based Auto-Pilot Interface for Drone to Object ChasingusingHistoricalTSPIData. 74-779. 10.23919/ICCAS59377.2023.10316981.

17. K. Xu, G. Zheng, and F. Zhang,“Deep Learning + Kalman Filter for Industrial Coil Center Tracking,”Metals Journal, 2022.

Publication Details

Chethana, G. M., Dhanush, M., Narein Karthik, E., Nithin, P., & Dhivya, R. (2025)"*Auto Track: Smart Aerial Object Tracking with Deep Learning".(IRJMETS)*, 7(9), 3368–3372. shttps://doi.org/10.56726/IRJMETS83167


IRJMETS
International Research Journal Of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
e-ISSN: 2582-5208
Ref: IRJMETS/Certificate/Volume 07/Issue 09/70900072870
DOI: https://www.doi.org/10.56726/IRJMETS83167
Date: 01/10/2025
Certificate of Publication
This is to certify that author "Chethana GM" with paper ID "IRJMETS70900072870" has published a paper entitled "AUTO TRACK: SMART AERIAL OBJECT TRACKING WITH DEEP LEARNING" in International Research Journal of Modernization in Engineering Technology and Science (IRJMETS), Volume 07, Issue 09, September 2025
Editor in Chief
IRJMETS Impact Factor 8.187
We Wish For Your Better Future
www.irjmets.com


IRJMETS
International Research Journal Of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
e-ISSN: 2582-5208
Ref: IRJMETS/Certificate/Volume 07/Issue 09/70900072870
DOI: https://www.doi.org/10.56726/IRJMETS83167
Date: 01/10/2025
Certificate of Publication
This is to certify that author "Nithin P" with paper ID "IRJMETS70900072870" has published a paper entitled "AUTO TRACK: SMART AERIAL OBJECT TRACKING WITH DEEP LEARNING" in International Research Journal of Modernization in Engineering Technology and Science (IRJMETS), Volume 07, Issue 09, September 2025
Editor in Chief
IRJMETS Impact Factor 8.187
We Wish For Your Better Future
www.irjmets.com


IRJMETS
International Research Journal Of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
e-ISSN: 2582-5208
Ref: IRJMETS/Certificate/Volume 07/Issue 09/70900072870
DOI: https://www.doi.org/10.56726/IRJMETS83167
Date: 01/10/2025
Certificate of Publication
This is to certify that author "Narein Karthik E" with paper ID "IRJMETS70900072870" has published a paper entitled "AUTO TRACK: SMART AERIAL OBJECT TRACKING WITH DEEP LEARNING" in International Research Journal of Modernization in Engineering Technology and Science (IRJMETS), Volume 07, Issue 09, September 2025
Editor in Chief
IRJMETS Impact Factor 8.187
We Wish For Your Better Future
www.irjmets.com


IRJMETS
International Research Journal Of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
e-ISSN: 2582-5208
Ref: IRJMETS/Certificate/Volume 07/Issue 09/70900072870
DOI: https://www.doi.org/10.56726/IRJMETS83167
Date: 01/10/2025
Certificate of Publication
This is to certify that author "Dhivya R" with paper ID "IRJMETS70900072870" has published a paper entitled "AUTO TRACK: SMART AERIAL OBJECT TRACKING WITH DEEP LEARNING" in International Research Journal of Modernization in Engineering Technology and Science (IRJMETS), Volume 07, Issue 09, September 2025
Editor in Chief
IRJMETS Impact Factor 8.187
We Wish For Your Better Future
www.irjmets.com


IRJMETS
International Research Journal Of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
e-ISSN: 2582-5208
Ref: IRJMETS/Certificate/Volume 07/Issue 09/70900072870
DOI: https://www.doi.org/10.56726/IRJMETS83167
Date: 01/10/2025
Certificate of Publication
This is to certify that author "Dhanush M" with paper ID "IRJMETS70900072870" has published a paper entitled "AUTO TRACK: SMART AERIAL OBJECT TRACKING WITH DEEP LEARNING" in International Research Journal of Modernization in Engineering Technology and Science (IRJMETS), Volume 07, Issue 09, September 2025
Editor in Chief
IRJMETS Impact Factor 8.187
We Wish For Your Better Future
www.irjmets.com

Appendix A – Code

Object Tracking

```
from djitellopy import tello import cv2 as opencv import keyboard
from time import sleep, time import datetime
import os
from ultralytics import YOLO import torch

# --- Tello and YOLO Setup --- me = tello.Tello() me.connect()
print(f"Battery life: {me.get_battery()}%") me.streamon()
# Load the YOLOv8 model.
model = YOLO('C:/Users/Narein karthik.E/Downloads/best (3).pt')

# --- GPU SETUP: Identify and set device --- if torch.cuda.is_available():
device = 'cuda'
print("Model will run on GPU (CUDA).") else:
device = 'cpu'
print("Model running on CPU (No CUDA device found).")

# Move the model weights to the selected device model.to(device)



# --- TARGET CONFIGURATION --- TARGET_CLASSES = ["Face"]

# --- DIAGNOSTIC: PRINT MODEL CLASS NAMES ---
try:
print(f"\n--- CUSTOM MODEL CLASS NAMES ---")
print(f"Your model contains {len(model.names)} classes: {model.names}") print(f"  \n")
except Exception as e:
print(f"Warning: Could not read model names. Error: {e}")

# --- Video Recording Setup ---
fourcc = opencv.VideoWriter_fourcc(*'mp4v') frame_width = 640
```

```
frame_height = 480 output_filename =
f"tello_recording_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.mp4" output_folder =
"tello_recordings"

if not os.path.exists(output_folder): os.makedirs(output_folder)

output_path = os.path.join(output_folder, output_filename) # Set VideoWriter FPS to 15 to match the
stable capture rate
out = opencv.VideoWriter(output_path, fourcc, 15, (frame_width, frame_height)) print(f"Recording video
to: {output_path}")
# --- Drone Control Variables --- is_tracking = False
fb_speed = 0
lr_speed = 0
ud_speed = 0
yv_speed = 0

# Proportional gain constants for the PID-like controller (SOFTENED FOR STABILITY) PID_P_YAW
= 0.20
PID_P_UD = 0.20
PID_P_FB = 0.08        # CRITICAL: Softest gain for stable distance control MAX_FB_SPEED = 35
# Target Area reduced to force the drone to move farther away TARGET_AREA = 35000

# ***VERTICAL OFFSET ADDED HERE (This is the negative value to force the drone to look
down)***
VERTICAL_OFFSET = -40
MIN_AREA_THRESHOLD = 5000
MAX_AREA_THRESHOLD = 150000
# DEAD ZONE INCREASED for tolerance FB_DEADBAND = 15000
# Function to get keyboard input and handle modes def get_keyboard_input():
global is_tracking, fb_speed, lr_speed, ud_speed, yv_speed
# Reset speeds
lr_speed, fb_speed, ud_speed, yv_speed = 0, 0, 0, 0
manual_override_speed = 50
```

```
# Toggle tracking mode
if keyboard.is_pressed("t"): is_tracking = not is_tracking
if is_tracking:
print("Tracking mode engaged.") else:
print("Tracking mode disengaged.") sleep(0.5)
# Check for manual override. If any manual key is pressed, disable tracking. if keyboard.is_pressed("left")
or keyboard.is_pressed("right") or
\
keyboard.is_pressed("up") or keyboard.is_pressed("down") or \ keyboard.is_pressed("w") or
keyboard.is_pressed("s") or \ keyboard.is_pressed("a") or keyboard.is_pressed("d"):
is_tracking = False
print("Manual override. Tracking disengaged.")

if not is_tracking:
# Manual control logic
if keyboard.is_pressed("left"): lr_speed = -manual_override_speed
elif keyboard.is_pressed("right"): lr_speed = manual_override_speed

if keyboard.is_pressed("up"): fb_speed = manual_override_speed
elif keyboard.is_pressed("down"): fb_speed = -manual_override_speed

if keyboard.is_pressed("w"): ud_speed = manual_override_speed
elif keyboard.is_pressed("s"): ud_speed = -manual_override_speed

if keyboard.is_pressed("a"):
yv_speed = -manual_override_speed elif keyboard.is_pressed("d"):
yv_speed = manual_override_speed

# Separate takeoff and land commands if keyboard.is_pressed("q"):
me.land() sleep(3)
if keyboard.is_pressed("e"): me.takeoff()
sleep(3)
def track_object(img):
global fb_speed, lr_speed, ud_speed, yv_speed
```

Gesture Control

```
import os
import tensorflow as tf import numpy as np
from tensorflow.keras.models import load_model import mediapipe as mp
import cv2 import threading import copy import time import djitellopy
from queue import Queue class events:
got_image = threading.Event() is_running = threading.Event() got_lms = threading.Event()
class VideoCaptureThread(threading.Thread): def _init_(
width=960, command_thresholds=None,
):
super(VideoCaptureThread, self).__init__()
if command_thresholds is None: command_thresholds = {
'up': 0, 'down': 0, 'left': 0, 'right': 0,
'forward': 0, 'backward': 0, 'flip': 0,
'land': 0, 'NaN': 0
}
self.video_source = video_source self.height = height
self.width = width self.tello_drone = tello_drone
# ---- Source selection: Tello camera or normal webcam ---- if self.video_source == 'tello' and
self.tello_drone is not None:
# Tello camera self.cap = None
self.frame_read = self.tello_drone.get_frame_read() self.frame = None
self.running = True else:
# Fallback: normal webcam
self.cap = cv2.VideoCapture(video_source) self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, self.height)
self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, self.width) self.frame = None
self.running = True
self.mpDraw = mp.solutions.drawing_utils self.lm_obj = None
self.lm_present = False self.mpHands = mp.solutions.hands self.labels = 'NaN'
self.font = cv2.FONT_HERSHEY_COMPLEX_SMALL self.landmark_queue =
Queue(20)self.command_handler = CommandHandler( self,
tello_drone=tello_drone, command_thresholds=command_threshol ds
```

```
)
self.gesture_detection_thread
.start()
self.artist = Artist(self, self.gesture_detection_threa
d) self.artist.start()
#---------- Video Recording
Setup ---- os.makedirs("TELLOGEST URE", exist_ok=True)
fourcc_mp4 = cv2.VideoWriter_fourcc(*"mp4v") filename_mp4 = os.path.join("TELLOGESTURE",
time.strftime("record_%Y%m%d_%H%M%S.mp4"))
self.video_writer = cv2.VideoWriter(filename_mp4, fourcc_mp4, 30.0, (self.width, self.height))
# --- FIX: Check if MP4 writer failed and fall back to MJPG (.avi) --- if not self.video_writer.isOpened():
print("WARNING: mp4 writer failed, falling back to AVI (MJPG codec).") # Using MJPG codec in an
AVI container for maximum compatibility fourcc_avi = cv2.VideoWriter_fourcc(*"MJPG")
filename_avi = os.path.join("TELLOGESTURE", time.strftime("record_%Y%m%d_%H%M%S.avi"))
self.video_writer = cv2.VideoWriter(filename_avi, fourcc_avi, 30.0, (self.width, self.height))
if self.video_writer.isOpened(): print(f"Recording video to: {filename_avi}")
else:
print("ERROR: VideoWriter failed to open. Recording disabled.") self.video_writer = None
else:
print(f"Recording video to: {filename_mp4}") # --- END FIX ---
self.start() def run(self):
# First frame
if self.cap is None:
self.frame = self.frame_read.frame
# Tello frame likely RGB -> convert to BGR for OpenCV self.frame = cv2.cvtColor(self.frame,
cv2.COLOR_RGB2BGR) self.frame = cv2.resize(self.frame, (self.width, self.height)) self.running =
True
else:
self.running, self.frame = self.cap.read()
while self.running: events.got_image.set()
self.frame = cv2.flip(self.frame, 1) self.mpDraw.draw_landmarks(self.frame, self.lm_obj,
self.mpHands.HAND_CONNECTIONS)
self.frame = cv2.flip(self.frame, 1) else:
```

```
self.lm_obj = self.landmark_queue.get() self.frame = cv2.flip(self.frame, 1)
self.mpDraw.draw_landmarks(self.frame, self.lm_obj, self.mpHands.HAND_CONNECTIONS)
self.frame = cv2.flip(self.frame, 1)
else:
self.labels = 'NaN'
cv2.putText(self.frame, f'Prediction : {self.labels}', (10, self.height - 20), self.font, 1, (255, 255, 255), 1, cv2.LINE_AA)
cv2.putText(self.frame, f'Drone Status : {self.drone_label}', (10, self.height - 40), self.font, 1, (255, 255, 255), 1, cv2.LINE_AA)
# ---- Write recording frame ---- if self.video_writer is not None:
# frame is already BGR, VideoWriter expects BGR self.video_writer.write(self.frame)
cv2.imshow("Original Frame", self.frame) key = cv2.waitKey(1) & 0xFF
# Quit
if key == ord('q'): self.stop() break
# Manual TAKEOFF if key == ord('t'):
if self.tello_drone is not None: print("Takeoff triggered by keyboard (T)") self.tello_drone.takeoff() self.put_drone_label("TAKEOFF")
# Manual LAND if key == ord('l'):
if self.tello_drone is not None: print("Landing triggered by keyboard (L)") self.tello_drone.send_rc_control(0, 0, 0, 0) self.tello_drone.land() self.put_drone_label("LANDING")
# Next frame
if self.cap is None:
self.frame = cv2.resize(self.frame, (self.width, self.height)) else:
self.running, self.frame = self.cap.read() self.stop()
def stop(self): self.running = False events.is_running.set()
if self.cap is not None: self.cap.release()
if hasattr(self, "video_writer") and self.video_writer is not None: self.video_writer.release()
print("Video recording saved.")
if self.tello_drone is not None: try:
self.tello_drone.streamoff() except Exception:
pass cv2.destroyAllWindows()
self.artist.end() self.command_handler.end() self.gesture_detection_thread.end()
```

self.artist.join() self.command_handler.join() self.gesture_detection_thread.join()

def put_drone_label(self, label): self.drone_label = label

def put_detection_label(self, label): self.labels = label

def put_landmarks(self, lm_obj=None, NaN=False): if NaN:

self.lm_present = False else:

self.landmark_queue.put(copy.copy(lm_obj)) self.lm_present = True

class GestureDetectionThread(threading.Thread):

self.class_names = np.array(

['backward', 'down', 'flip', 'forward', 'land', 'left', 'right', 'up'], dtype=object

)

Model file must be in same folder as this script self.model = load_model('Model_5_120e.h5')

self.lm_obj = None self.video_capture = video_capture self.direction = 'NaN'

self.running = True

self.NaN_threshold = 4 self.cur_NaN_threshold = self.NaN_threshold

self.command_handler = command_handler def run(self):

while self.running:

events.got_lms.wait() if not self.running:

break self.hand_detection()

def end(self): self.running = False events.got_lms.set()

def hand_detection(self):

=== 1) Relative coordinates w.r.t wrist (landmark 0) === landmarks = self.lm_obj.landmark

base_x = landmarks[0].x base_y = landmarks[0].y

coords = []

for lm in landmarks: rel_x = lm.x - base_x rel_y = lm.y - base_y coords.append(rel_x) coords.append(rel_y)

flat = np.array(coords, dtype=np.float32).re shape(1, 42)

=== 2) Normalize by max absolute value === max_value = np.max(np.abs(flat))

if max_value > 0:

prediction = self.model.predict(flat, verbose=0) predicted_idx = int(np.argmax(prediction))

self.direction = self.class_names[predicted_idx] # Debug in console

print(f"Raw output: {prediction[0]}")

print(f"Predicted index: {predicted_idx}, label: {self.direction}")

self.video_capture.put_detection_label(self.direction) self.command_handler.send_commands(self.direction)

Appendix-B Photos